

**TRƯỜNG ĐẠI HỌC SÀI GÒN  
KHOA CÔNG NGHỆ THÔNG TIN**

-----\*\*\*-----



**BÁO CÁO ĐỒ ÁN CUỐI KỲ**

**Môn học: Seminar chuyên đề**

**XÂY DỰNG TRANSFORMER CHO BÀI TOÁN  
PHÂN LOẠI CẢM XÚC VĂN BẢN**

**Giảng viên hướng dẫn: PGS.TS. Nguyễn Tuấn Đăng**

**Nhóm sinh viên thực hiện:**

Nguyễn Khánh Văn 3122410463

Nguyễn Thành Khang 3122410172

*Thành phố Hồ Chí Minh, tháng 5 năm 2026*

## Nhận xét của giảng viên

This image shows a full page of primary-ruled paper. It features approximately 20 horizontal dotted lines spaced evenly down the page, providing a guide for handwriting practice. The background is white, and there are no margins or additional markings.

# Mục lục

|                                                                |    |
|----------------------------------------------------------------|----|
| Lời mở đầu .....                                               | 1  |
| Lời cảm ơn .....                                               | 2  |
| Chương 1: Giới thiệu bài toán.....                             | 3  |
| 1.1. Định nghĩa bài toán .....                                 | 3  |
| 1.2. Đặc điểm tập dữ liệu .....                                | 4  |
| 1.3. Mục tiêu học thuật.....                                   | 5  |
| 1.4. Phạm vi cài đặt .....                                     | 7  |
| Chương 2: Kiến trúc và Cài đặt mô hình .....                   | 8  |
| 2.1. Mô tả kiến trúc mô hình .....                             | 8  |
| 2.1.1. Luồng dữ liệu tổng thể (Data Flow) .....                | 8  |
| 2.1.2. Vai trò của từng thành phần trong mô hình.....          | 9  |
| 2.2. Chi tiết Cài đặt Self-Attention và FFN/EncoderBlock ..... | 12 |
| 2.2.1. Cài đặt hàm Scaled Dot-Product Attention .....          | 12 |
| 2.2.2. Cài đặt mạng (FFN): .....                               | 13 |
| 2.2.3. Tích hợp thành Transformer Encoder Block .....          | 14 |
| Chương 3: Thực nghiệm và Đánh giá kết quả.....                 | 15 |
| 3.1. Thiết lập thực nghiệm.....                                | 15 |
| 3.1.1. Quy trình thực thi hệ thống .....                       | 15 |
| 3.1.2. Cấu hình siêu tham số (Hyperparameters) .....           | 15 |
| 3.1.3. Mô hình cơ sở (Baseline Model).....                     | 16 |
| 3.1.4. Đảm bảo tính tái lập kết quả (Reproducibility) .....    | 16 |
| 3.2. Kết quả thực nghiệm và so sánh .....                      | 17 |
| 3.2.1. Bảng tổng hợp kết quả thực nghiệm.....                  | 17 |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 3.2.2. Đánh giá và lựa chọn cấu hình tốt nhất .....                               | 17 |
| 3.2.3. Đồ thị quá trình huấn luyện (Learning Curve) .....                         | 18 |
| 3.2.4. Phân tích hiện tượng Overfitting (Quá khớp).....                           | 22 |
| 3.2.5. Nhận xét về tác động của các siêu tham số ( $d_{model}$ và $d_{ff}$ )..... | 23 |
| 3.3. Phân tích Attention.....                                                     | 24 |
| 3.3.1. Phân tích 3 trường hợp.....                                                | 24 |
| 3.3.2. Nhận xét tổng quát .....                                                   | 28 |
| 3.4. Error Analysis .....                                                         | 28 |
| 3.4.1. Bảng thống kê các mẫu phân loại sai .....                                  | 28 |
| 3.4.2. Phân nhóm nguyên nhân gây lỗi .....                                        | 29 |
| 3.4.3. Đề xuất hướng cải thiện .....                                              | 30 |
| Chương 4: Kết luận .....                                                          | 31 |
| 4.1. Tóm tắt những kết quả đạt được.....                                          | 31 |
| 4.2. Bài học quan trọng rút ra từ quá trình tự cài đặt Self-Attention.....        | 31 |
| 4.3. Giới hạn của mô hình trên bộ dữ liệu nhỏ .....                               | 32 |
| Tài liệu tham khảo.....                                                           | 33 |
| Phụ lục.....                                                                      | 34 |

## Danh mục hình ảnh

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Hình 2.1: Sơ đồ tổng thể .....                                                | 9  |
| Hình 3.1: Đồ thị quá trình huấn luyện theo mô hình Baseline MLP.....          | 18 |
| Hình 3.2: Đồ thị quá trình huấn luyện theo mô hình Transformer #1.....        | 19 |
| Hình 3.3: Đồ thị quá trình huấn luyện theo mô hình Transformer #2.....        | 20 |
| Hình 3.4: Đồ thị quá trình huấn luyện theo mô hình Transformer #3.....        | 21 |
| Hình 3.5: Heatmap “we discussed the movie in class at home” .....             | 24 |
| Hình 3.6: Heatmap “the film was pretty wonderful today” .....                 | 25 |
| Hình 3.7: Heatmap “this movie is not amazing” .....                           | 26 |
| Hình 3.8: Heatmap “the acting is never bad” .....                             | 27 |
| Hình Phụ lục 1: Giao diện web chạy thử nghiệm mô hình .....                   | 35 |
| Hình Phụ lục 2: Giao diện web lịch sử phân tích thử nghiệm.....               | 35 |
| Hình Phụ lục 3: Giao diện xem cơ sở dữ liệu .....                             | 36 |
| Hình Phụ lục 4: Giao diện thống kê các thực nghiệm đã dùng trong lịch sử..... | 36 |
| Hình Phụ lục 5: Giao diện thay đổi mô hình thực nghiệm.....                   | 37 |
| Hình Phụ lục 6: Giao diện kết quả sau khi phân tích .....                     | 37 |

## Danh mục bảng biểu

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Bảng 1: Kết quả chi tiết sau quá trình huấn luyện.....                   | 17 |
| Bảng 2: Tổng hợp câu văn tiêu biểu mà mô hình đã dự đoán không chính xác | 28 |

# Danh mục từ viết tắt

**FFN:** FeedForward Network (Mạng lan truyền xuôi).

**MLP:** Multi-Layer Perceptron (Mạng nơ-ron truyền thẳng), được sử dụng làm mô hình cơ sở (Baseline).

**OOV:** Out-Of-Vocabulary (Từ vựng lạ, tức là những từ xuất hiện trong tập kiểm tra nhưng chưa từng gặp trong tập huấn luyện).

**Q, K, V:** Các ma trận cốt lõi trong cơ chế Attention, lần lượt là Query (Truy vấn), Key (Khóa), và Value (Giá trị).

**Acc:** Accuracy (Độ chính xác), thường được dùng trong các bảng biểu và đồ thị (ví dụ: Train Acc, Val Acc, Test Acc).

**PAD:** Padding (Ký tự/token đệm được thêm vào để làm đồng đều độ dài các câu).

**STT:** Số thứ tự.

**MSSV:** Mã số sinh viên.

**nn:** Tiền tố đại diện cho Neural Network trong các module của thư viện PyTorch (ví dụ: nn.MultiheadAttention, nn.LayerNorm).

**IDs:** Identifiers (Định danh), dùng trong "Token IDs" để chỉ các từ sau khi được tiền xử lý thành các giá trị rời rạc.

# Lời mở đầu

Đề án "Xây dựng Transformer cho bài toán phân loại cảm xúc văn bản" tập trung vào việc nghiên cứu và ứng dụng kiến trúc Transformer - một trong những mô hình học sâu cốt lõi trong lĩnh vực xử lý ngôn ngữ tự nhiên. Bài toán cụ thể được đặt ra là phân loại các câu văn tiếng Anh ngắn (độ dài tối đa 20 từ) vào ba lớp cảm xúc độc lập: Tích cực (Positive), Tiêu cực (Negative) và Trung tính (Neutral). Quá trình huấn luyện và đánh giá mô hình được thực hiện trên tập dữ liệu gồm 600 mẫu, được phân chia theo tỷ lệ 420/90/90 đối với các tập huấn luyện, kiểm định và kiểm tra.

Mục tiêu trọng tâm của nghiên cứu này là đi sâu vào cơ chế hoạt động của khối kiến trúc Self-Attention. Thông qua việc tự cài đặt thủ công các thành phần toán học cốt lõi, nghiên cứu hướng đến việc đánh giá năng lực biểu diễn của mô hình và giải thích (interpretability) các kết quả dự đoán thông qua phân tích ma trận trọng số chú ý.

Để đảm bảo tính minh bạch, phạm vi cài đặt của hệ thống được phân định rõ ràng. Các thành phần nền tảng bao gồm Scaled Dot-Product Attention và FFN được lập trình hoàn toàn thông qua các phép toán tensor cơ bản. Các thư viện hỗ trợ (PyTorch) chỉ được sử dụng cho các module phụ trợ như LayerNorm, Positional Encoding, và Tokenization nhằm tối ưu hóa quy trình thực nghiệm.

Trân trọng!

## Lời cảm ơn

Nhóm thực hiện xin gửi lời cảm ơn đến PGS.TS. Nguyễn Tuấn Đăng đã cung cấp tài liệu hướng dẫn, khung mã nguồn cơ sở và định hướng học thuật để nhóm có thể hoàn thành chuyên đề nghiên cứu này.



# Chương 1: Giới thiệu bài toán

## 1.1. Định nghĩa bài toán

Về bản chất, phân loại cảm xúc văn bản (Sentiment Classification) là bài toán phân loại chuỗi (Sequence Classification) cốt lõi trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên (NLP). Đề án này tập trung xây dựng một hàm ánh xạ phi tuyến tính thông qua kiến trúc mạng Transformer Encoder tinh gọn, nhằm chuyển đổi thông tin từ không gian chuỗi rời rạc sang không gian phân phối xác suất của các nhãn cảm xúc. Bài toán được định dạng chặt chẽ như sau:

### a. Đầu vào (Input)

Dữ liệu đầu vào không dừng lại ở văn bản thô, mà phải trải qua quá trình tiền xử lý để chuyển thành các chuỗi định danh từ vựng (Token IDs)

- Giới hạn chuỗi: Mỗi câu văn tiếng Anh được chuẩn hóa về một chiều dài cố định là  $max\_len = 20$  từ. Đối với các câu có độ dài chênh lệch, hệ thống bắt buộc áp dụng kỹ thuật cắt tĩa (Truncation) hoặc đệm thêm các ký tự rỗng (Padding bằng token [PAD]) để làm đồng đều dữ liệu.
- Chiều dữ liệu (Shape): Khi đưa vào mô hình để huấn luyện, một lô dữ liệu đầu vào sẽ được biểu diễn dưới dạng Tensor 2 chiều có kích thước  $(batch\_size, 20)$ .

### b. Đầu ra (Output)

Mô hình không trả về chữ viết trực tiếp, mà xuất ra một vector đại diện cho phân phối xác suất của câu văn đối với các cực tính cảm xúc.

- Chiều dữ liệu (Shape): Đầu ra cuối cùng sau khi đi qua khối mạng phân lớp (Classifier Head) là một Tensor kích thước  $(batch\_size, 3)$ .
- Không gian nhãn: Giá trị lớn nhất trong 3 node đầu ra này sẽ đóng vai trò quyết định câu văn thuộc về một trong ba lớp cảm xúc độc lập, bao gồm: Tích cực (Positive), Tiêu cực (Negative), và Trung tính (Neutral).

## 1.2. Đặc điểm tập dữ liệu

### a) Nguồn gốc và quy mô tập dữ liệu

Việc mô hình được huấn luyện trên một tập dữ liệu giả lập có quy mô vi mô (chỉ 600 câu tiếng Anh ngắn với độ dài tối đa 20 từ) mang ý đồ học thuật rất rõ. Thay vì chạy theo mục tiêu đạt độ chính xác (accuracy) cao nhất như các mô hình ngôn ngữ lớn, quy mô dữ liệu này được thiết kế để sinh viên tập trung thấu hiểu sâu sắc cơ chế toán học của khối Self-Attention. Kích thước nhỏ gọn giúp tối ưu hóa thời gian huấn luyện và tài nguyên tính toán cho hệ thống.

Tuy nhiên, vì Transformer vốn là một kiến trúc mạng "khao khát dữ liệu" (data-hungry), việc chỉ có 600 mẫu cũng tạo ra những giới hạn và thách thức thực nghiệm đáng giá, điển hình là việc mô hình dễ rơi vào trạng thái "học vẹt" (overfitting) ghi nhớ các mẫu thay vì quy luật tổng quát nếu cấu hình quá lớn, hoặc gặp khó khăn với các lỗi từ vựng lạ (Out-Of-Vocabulary - OOV) do không gian từ điển bị hạn chế.

### b) Tính phân bố dữ liệu

Đặc điểm cân bằng hoàn hảo cho mỗi lớp cảm xúc (Tích cực- Positive, Tiêu cực - Negative, Trung tính - Neutral) là một thiết lập lý tưởng và thiết yếu trong học máy. Sự phân bố đồng đều này giúp giải quyết triệt để vấn đề thiên lệch dữ liệu (bias).

Trong các bộ dữ liệu mất cân bằng, mô hình thường có xu hướng "gian lận" bằng cách dự đoán nghiêng về nhãn xuất hiện nhiều nhất để kéo cao điểm số. Nhờ tính cân bằng tuyệt đối này, mọi chỉ số đánh giá độ chính xác (Accuracy) đều phản ánh trung thực năng lực biểu diễn của kiến trúc mạng, đảm bảo cơ chế Attention đang thực sự học cách trích xuất các đặc trưng ngữ nghĩa phức tạp một cách công bằng cho cả ba sắc thái cảm xúc.

### c) Chiến lược phân chia tập dữ liệu

Việc bộ dữ liệu được chia tách cứng ngay từ đầu thành 3 tập riêng biệt giúp chuẩn hóa và đảm bảo tính khách quan cho quy trình đánh giá thực nghiệm.

- Tập huấn luyện (Train - 420 mẫu, chiếm 70%): Dùng trực tiếp để mô hình lan truyền xuôi, tính toán sai số và cập nhật trọng số. Với số lượng vốn vụn 420 câu, nó trở thành phép thử khắc nghiệt nhất để quan sát ranh giới giữa việc mô hình có khả năng tổng quát hóa hay bị "quá khớp" (overfitting).
- Tập kiểm định (Validation - 90 mẫu, chiếm 15%): Đóng vai trò là tập giám sát độc lập trong lúc huấn luyện. Nó giúp theo dõi hiệu năng qua từng vòng lặp (epoch), kiểm soát hiện tượng quá khớp thông qua biểu đồ Learning Curve, và là thước đo để người nghiệm lựa chọn ra cấu hình siêu tham số (hyperparameter) tối ưu nhất.
- Tập kiểm tra (Test - 90 mẫu, chiếm 15%): Được cất giữ hoàn toàn bí mật, không tham gia vào bất kỳ quá trình huấn luyện hay tinh chỉnh nào, tập dữ liệu này có nhiệm vụ phơi bày năng lực thực chiến cuối cùng của mô hình trước các dữ liệu hoàn toàn mới lạ một cách minh bạch và khách quan nhất.

## 1.3. Mục tiêu học thuật

### a) Từ bỏ cuộc đua tối ưu hóa độ chính xác (Accuracy)

Khác với các dự án ứng dụng thực tế thường có xu hướng sử dụng các mô hình khổng lồ đã được huấn luyện sẵn (như là Pre-trained LLMs) để cố gắng nâng cao điểm số lên cao nhất, bài toán này chủ động giới hạn không gian thực nghiệm.

Do mô hình được đặt trong một môi trường vi mô với bộ dữ liệu giả lập chỉ vốn vụn 600 mẫu, việc đạt độ chính xác gần như tuyệt đối (ví dụ 99% trên tập Train) thực chất chỉ phơi bày hiện tượng mô hình đang "học vẹt" (overfitting). Vì kiến trúc Transformer mang bản chất khao khát dữ liệu (data-hungry), độ chính xác cao trên tập dữ liệu này không phản ánh năng lực tổng quát hóa thực tế.

Do đó, thay vì tinh chỉnh siêu tham số chỉ để lấy điểm số đẹp, việc thực nghiệm nhằm mục đích quan sát ranh giới giới hạn của mô hình, giúp nhận diện điểm cân bằng lý tưởng (sweet spot) giữa sức chứa (capacity) của mạng và nguy cơ quá khớp.

## b) Giải phẫu chi tiết cơ sở toán học của Self-Attention

Để thực sự hiểu sâu sắc cơ chế hoạt động, đồ án đặt ra một ranh giới kỹ thuật vô cùng nghiêm ngặt: tuyệt đối không được dùng mô hình như một "hộp đen" thông qua các hàm tự động có sẵn của thư viện (điển hình là `nn.MultiheadAttention`). Thay vào đó, sinh viên bắt buộc phải tự lập trình thủ công (from scratch) từ những phép toán tensor cơ bản nhất.

Quá trình tự xây này buộc người học phải thấu hiểu chi tiết luồng chảy dữ liệu (tensor flow), nắm rõ sự dịch chuyển kích thước của các ma trận Truy vấn (Q), Khóa (K), Giá trị (V) qua từng lớp tính toán.

Quan trọng hơn, nó giúp người học trực tiếp nghiệm chứng các lý thuyết toán học lỗi, chẳng hạn như hiểu được tác dụng sống còn của việc chia điểm số chú ý cho hệ số tỉ lệ  $\left(\frac{1}{\sqrt{d_k}}\right)$  nhằm cứu vãn dòng chảy đạo hàm khỏi hiện tượng triệt tiêu (vanishing gradient) trước khi đưa qua hàm Softmax.

## c) Phát triển năng lực diễn dịch và phân tích thực nghiệm

Mục tiêu học thuật cuối cùng được hiện thực hóa thông qua việc rèn luyện kỹ năng phân tích và giải thích quyết định của mạng nơ-ron - một khía cạnh cực kỳ quan trọng của **Trí tuệ nhân tạo** có thể giải thích. Từ ma trận trọng số thu được, sinh viên phải trực quan hóa chúng thành các biểu đồ nhiệt (Attention Heatmaps) để biến các phép toán trừu tượng thành bằng chứng trực quan rõ nét.

Thông qua các kịch bản kiểm thử (đúng, sai, hoặc chứa cấu trúc phủ định như "not", "never"), người nghiệm có thể nhìn thấu vào "suy nghĩ" của mô hình, xem nó đang tập trung sự chú ý vào các từ khóa mang sắc thái cảm xúc cốt lõi, hay đang bị đánh lừa bởi các từ vựng nhiễu. Kết hợp với việc phân tích lỗi (Error Analysis) và so sánh chéo với mô hình nền (Baseline MLP), quá trình này mài giũa tư duy phản biện, giúp người học biết cách "bắt bệnh", giải thích nguyên nhân và đề xuất hướng nâng cấp mô hình một cách khoa học.

## 1.4. Phạm vi cài đặt

### a. Thành phần bắt buộc tự cài đặt

Cơ chế Scaled Dot-Product Attention: Phải tự lập trình công thức tính điểm số chú ý (Attention Scores) bằng cách nhân ma trận Query (Q) với ma trận chuyển vị của Key (K), chia cho hệ số tỷ lệ  $\sqrt{d_k}$ , áp dụng Softmax và cuối cùng nhân với ma trận Value (V). Tuyệt đối không sử dụng hàm có sẵn `nn.MultiheadAttention` của PyTorch.

Mạng FFN: Phải tự nối 2 lớp biến đổi tuyến tính (được phép dùng `nn.Linear` để khởi tạo) với một hàm kích hoạt phi tuyến tính ReLU ở giữa.

### b. Thành phần được phép sử dụng thư viện

Add & LayerNorm (Cơ chế cộng và chuẩn hóa): Được phép gọi trực tiếp module `nn.LayerNorm` của PyTorch.

Positional Encoding (Mã hóa vị trí) & Classifier Head: Các phần này đã có sẵn code mẫu do giảng viên cung cấp, bạn không cần tự lập trình lại công thức lượng giác sin/cos.

Các hàm tối ưu và đánh giá: Được gọi thư viện PyTorch cho `nn.Embedding` (lớp nhúng), Adam (thuật toán tối ưu) và `CrossEntropy` (hàm mất mát).

Tiền xử lý và trực quan hóa: Các bước đọc dữ liệu, phân mảnh từ (Tokenization) hoặc vẽ biểu đồ chú ý (Visualization) được tự do sử dụng bất kỳ thư viện nào (như thư viện làm sẵn trong `data_utils.py` và `visualize.py`).

## Chương 2: Kiến trúc và Cài đặt mô hình

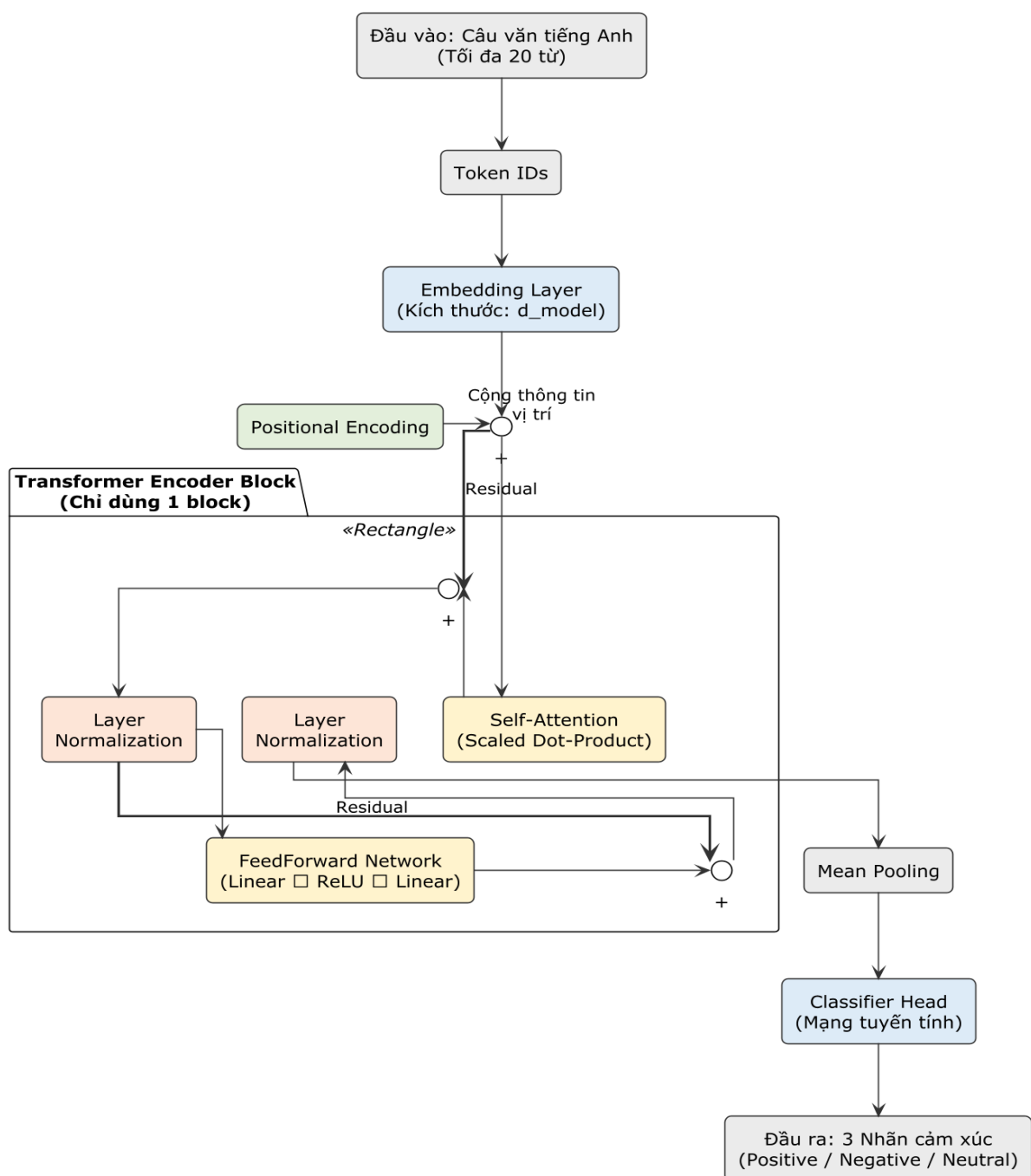
### 2.1. Mô tả kiến trúc mô hình

#### 2.1.1. Luồng dữ liệu tổng thể (Data Flow)

Mô hình được xây dựng dựa trên kiến trúc Transformer Encoder tinh gọn, luồng dữ liệu bắt đầu bằng quá trình tiền xử lý các câu văn tiếng Anh ngắn (giới hạn tối đa 20 từ) thành một chuỗi các Token IDs rồi raster mang kích thước ma trận là  $(batch\_size, seq\_len)$ . Chuỗi định danh này sau đó được truyền qua lớp Embedding để ánh xạ thành các vector số học liên tục trong một không gian có số chiều là  $d\_model$ . Bởi vì cơ chế của Transformer xử lý song song và không tự mang tính tuần tự, chuỗi vector này sẽ được cộng thêm tín hiệu vị trí từ lớp Positional Encoding (sử dụng các hàm lượng giác sin/cos), tạo ra một khối dữ liệu biểu diễn hoàn chỉnh có kích thước  $(batch\_size, seq\_len, d\_model)$  trước khi đi vào mạng nơ-ron.

Tiếp đó, khối dữ liệu biểu diễn này đi qua một khối Transformer Encoder Block duy nhất. Tại đây, dữ liệu đi qua lớp Self-Attention, nơi ma trận Q, K và V tương tác với nhau để tính toán điểm tương quan chéo giữa mọi cặp từ, từ đó trích xuất ra đặc trưng ngữ cảnh phức tạp của câu. Sau đó, luồng dữ liệu tiếp tục đi qua mạng lan truyền xuôi FFN với hai lớp tuyến tính đan xen hàm ReLU để khai phá thêm các đặc tính phi tuyến tính. Khép lại mỗi tiểu lớp (Self-Attention và FFN), hệ thống đều áp dụng cơ chế cộng thẳng dư (Residual Connection) và chuẩn hóa (Layer Normalization) giúp thông tin lưu thông xuyên suốt mà không bị triệt tiêu đạo hàm, giữ nguyên kích thước đầu ra của khối là  $(batch\_size, seq\_len, d\_model)$ .

Ở giai đoạn cuối, đầu ra của khối Encoder là một ma trận chứa biểu diễn vector của từng từ riêng lẻ, nên luồng dữ liệu sẽ được thu gọn lại thông qua bước Mean Pooling. Quá trình này tính trung bình cộng tất cả các vector dọc theo chiều dài chuỗi (sequence length) để nén dữ liệu thành một vector ngữ nghĩa duy nhất đại diện cho toàn bộ câu văn, với kích thước thu hẹp còn  $(batch\_size, d\_model)$ . Cuối cùng, vector đại diện duy nhất này được đưa qua mạng phân lớp Classifier Head để thực hiện phép biến đổi tuyến tính, ánh xạ thẳng thành 3 node dự đoán có kích thước  $(batch\_size, 3)$ , tương ứng với xác suất câu văn thuộc về 3 nhãn cảm xúc: Tích cực (*Positive*), Tiêu cực (*Negative*) và Trung tính (*Neutral*).



Hình 2.1: Sơ đồ tổng thể

## 2.1.2. Vai trò của từng thành phần trong mô hình

### a. Embedding (Lớp nhúng)

Đóng vai trò là cửa ngõ của mạng nơ-ron, lớp Embedding thực hiện nhiệm vụ ánh xạ các từ (đang ở dạng Token IDs rời rạc sau quá trình tiền xử lý) thành các vector số học liên tục. Quá trình này biến đổi luồng dữ liệu thành một khối ma trận không gian thực có số chiều được xác định bởi siêu tham số  $d_{model}$ .

Sự chuyển đổi này không chỉ đơn thuần là thay đổi định dạng dữ liệu mà còn mang ý nghĩa cực kỳ quan trọng về mặt biểu diễn ngôn ngữ. Thông qua quá trình huấn luyện, lớp nhúng này sẽ tự động điều chỉnh các trọng số để đưa các vector biểu diễn của những từ có ngữ nghĩa tương đồng lại gần nhau hơn trong không gian vector, tạo nên tảng vững chắc cho việc phân tích ngữ cảnh ở các bước tiếp theo.

### **b. Positional Encoding (Mã hóa vị trí)**

Bản chất của kiến trúc Transformer là cơ chế Self-Attention xử lý tất cả các từ trong câu cùng một lúc (song song) thay vì đọc từng từ một theo thứ tự như các mạng RNN truyền thống. Do đặc tính này, mô hình hoàn toàn không tự nhận biết được tính tuần tự, dẫn đến việc đảo lộn trật tự từ có thể không làm thay đổi kết quả tính toán nếu không có sự can thiệp bổ sung.

Để khắc phục nhược điểm này, mô hình sử dụng Positional Encoding (được tính toán qua các hàm lượng giác sin/cos) để sinh ra một tín hiệu vị trí và cộng trực tiếp tín hiệu đó vào ma trận Embedding. Sự kết hợp này giúp mô hình không chỉ hiểu được ngữ nghĩa của từng từ mà còn nhận biết chính xác vị trí, thứ tự trước sau của các từ, từ đó nắm bắt được trật tự ngữ pháp trong câu văn.

### **c. Transformer Encoder Block (Khối mã hóa)**

Đây là trái tim của mạng nơ-ron giúp phân tích ngữ nghĩa. Trong đồ án này, kiến trúc được thiết kế tối giản khi chỉ sử dụng 1 Transformer Encoder block. Lý do là vì bộ dữ liệu thực nghiệm có kích thước rất nhỏ (chỉ 600 mẫu), việc giới hạn ở 1 block giúp mô hình đủ nhẹ để học nhanh, đồng thời tránh được hiện tượng học vẹt (overfitting) so với việc xếp chồng nhiều lớp phức tạp không cần thiết. Khối block này chứa các thành phần:

- **Self-Attention:** Giúp mô hình tính toán trọng số chú ý của mỗi từ lên tất cả các từ còn lại trong câu, từ đó liên kết các từ lại với nhau để nắm bắt ngữ cảnh cốt lõi.
- **FFN:** Là một mạng lan truyền xuôi gồm hai lớp tuyến tính (Linear) với hàm kích hoạt ReLU ở giữa, đóng vai trò chuyển đổi phi tuyến tính để học ra các đặc trưng sâu hơn sau khi đã lấy được thông tin từ Attention.



- **Add & LayerNorm (Residual Connection và Layer Normalization):**

Được áp dụng ngay sau mỗi tiểu lớp (sublayer) Self-Attention và FFN. Việc sử dụng Residual Connection (Add) cho phép cộng trực tiếp dữ liệu đầu vào với đầu ra của tiểu lớp, giúp dòng chảy thông tin (gradient) đi qua dễ dàng hơn, ngăn ngừa hiện tượng triệt tiêu đạo hàm. Trong khi đó, Layer Normalization chuẩn hóa dữ liệu trên từng đặc trưng, giúp quá trình huấn luyện diễn ra ổn định và hội tụ nhanh chóng hơn

**d. Mean Pooling (Thu gọn trung bình)**

Sau khi đi qua khối Transformer Encoder, đầu ra nhận được vẫn là một ma trận đa chiều biểu diễn thông tin ngữ cảnh của từng từ riêng lẻ dọc theo chiều dài chuỗi (sequence length). Tuy nhiên, để thực hiện bài toán phân loại cho toàn bộ câu, hệ thống cần một vector đại diện duy nhất chứ không phải là một tập hợp các vector từ vựng.

Để giải quyết vấn đề này, bước Mean Pooling có nhiệm vụ tính trung bình cộng tất cả các vector đại diện của mọi từ trong câu văn đó. Quá trình này đóng vai trò như một bộ nén thông tin, gom tụ toàn bộ đặc trưng ngữ cảnh phức tạp lại thành một vector ngữ nghĩa duy nhất và toàn diện, mang kích thước ( $batch\_size, d\_model$ ), đại diện cho cảm xúc chung của cả câu.

**e. Classifier Head (Mạng phân lớp)**

Là thành phần cuối cùng trong luồng dữ liệu tổng thể, đóng vai trò đưa ra quyết định phân loại. Khối này nhận đầu vào là vector đại diện duy nhất vừa được tổng hợp từ bước Mean Pooling và đưa nó đi qua một lớp biến đổi tuyến tính (Linear Layer) cuối cùng.

Nhiệm vụ của phép biến đổi tuyến tính này là ánh xạ không gian đặc trưng từ số chiều  $d\_model$  thành đúng 3 node dự đoán. Đầu ra của 3 node này tương ứng với xác suất dự đoán câu văn thuộc về 3 lớp cảm xúc độc lập đã được định nghĩa trong bài toán: Tích cực (Positive), Tiêu cực (Negative) và Trung tính (Neutral).

## 2.2. Chi tiết Cài đặt Self-Attention và FFN/EncoderBlock

### 2.2.1. Cài đặt hàm Scaled Dot-Product Attention

Đây là thành phần cốt lõi nhất của đồ án và bắt buộc phải được tự cài đặt thủ công thông qua các phép toán tensor cơ bản, nghiêm cấm việc sử dụng hàm có sẵn như `nn.MultiheadAttention`. Đây là code đã sử dụng trong hệ thống:

```
def scaled_dot_product_attention(Q, K, V):
    """
    Q, K, V: shape (batch_size, seq_len, d_k)
    Return:
        output : shape (batch_size, seq_len, d_k)
        weights : shape (batch_size, seq_len, seq_len)
    """
    d_k = Q.size(-1)

    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.
    weights = F.softmax(scores, dim=-1)
    output = torch.matmul(weights, V)

    return output, weights
```

Hàm *scaled\_dot\_product\_attention* được xây dựng để thực hiện các bước sau:

- **Đầu vào:** Nhận vào 3 ma trận Query ( $Q$ ), Key ( $K$ ), và Value ( $V$ ), mỗi ma trận đều có kích thước là  $(batch\_size, seq\_len, d_k)$ .
- **Tính Attention Scores:** Điểm chú ý (scores) được tính toán bằng phép nhân ma trận giữa  $Q$  (Ma trận truy vấn) và ma trận chuyển vị của  $K$  ( $QK^T$ ), sau đó bắt buộc phải chia cho một hệ số tỉ lệ là  $d_k$  (scaling). Công thức toán học là:  $scores = \left( \frac{QK^T}{\sqrt{d_k}} \right)$ . Việc chia cho hệ số tỉ lệ này đóng vai trò rất quan trọng để tránh hiện tượng mất đạo hàm (vanishing/exploding gradient) trong quá trình huấn luyện khi số chiều  $d_k$  lớn. Ma trận đầu ra ở bước này có kích thước  $(batch\_size, seq\_len, seq\_len)$ .
- **Tính Attention Weights:** Áp dụng hàm Softmax dọc theo chiều cuối cùng ( $dim=-1$ ) của ma trận scores để chuẩn hóa thành các trọng số chú ý có giá trị từ 0 đến 1. Việc chuẩn hóa này đảm bảo tổng các trọng số chú ý của một từ lên các từ khác luôn xấp xỉ bằng 1.0.

- **Tính Context Vector:** Cuối cùng, nhân ma trận trọng số (attention weights) vừa thu được với ma trận V để tạo ra context vector mang thông tin ngữ cảnh tổng hợp, với kích thước trả về là  $(batch\_size, seq\_len, d_k)$ .

### 2.2.2. Cài đặt mạng (FFN):

Cùng với Self-Attention, lớp FeedForwardNetwork cũng là một thành phần bắt buộc tự cài đặt. Mạng này đóng vai trò tăng cường khả năng biểu diễn phi tuyến tính của mô hình sau khi thu thập thông tin ngữ cảnh. Công thức toán học là:

$$FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

Kiến trúc của FFN bao gồm 2 lớp biến đổi tuyến tính (Linear layers) kết hợp với một hàm kích hoạt phi tuyến tính ReLU ở giữa. Quá trình biến đổi kích thước ma trận diễn ra cụ thể như sau:

- Đầu tiên, dữ liệu đi qua lớp tuyến tính thứ nhất: biến đổi kích thước từ số chiều mặc định  $Linear(d\_model, d\_ff)$  để mở rộng sang không gian chiều ẩn lớn hơn.
- Dữ liệu tiếp tục đi qua hàm kích hoạt ReLU.
- Cuối cùng, dữ liệu đi qua lớp tuyến tính thứ hai: thu hẹp kích thước trở lại ban đầu qua phép biến đổi  $Linear(d\_ff, d\_model)$ .

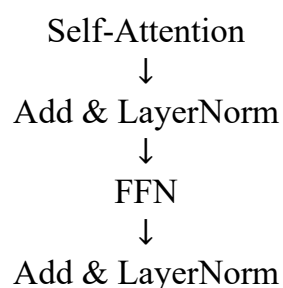
Đây là code đã được sử dụng trong hệ thống:

```
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model: int, d_ff: int):
        super().__init__()
        # Linear(d_model, d_ff) -> ReLU -> Linear(d_ff, d_model)
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        # Forward pass của FFN
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        return x
```

### 2.2.3. Tích hợp thành Transformer Encoder Block

Khối Transformer Encoder Block là thành phần trung tâm của kiến trúc Transformer, nơi tích hợp các module cốt lõi thành một quy trình lan truyền xuôi thông nhất. Dữ liệu được xử lý tuần tự qua hai tiểu lớp chính gồm Self-Attention và Feed-Forward Network (FFN), xen giữa là cơ chế Add & LayerNorm. Cấu trúc này giúp tensor được tinh chỉnh liên tục, làm giàu ngữ nghĩa và đảm bảo khả năng học đặc trưng hiệu quả qua từng bước tính toán. Luồng xử lý diễn ra theo thứ tự:



Ở tiểu lớp đầu tiên, dữ liệu đầu vào chứa thông tin nhúng và vị trí được đưa vào cơ chế Self-Attention để tính toán mức độ tương quan giữa các từ, từ đó trích xuất đặc trưng ngữ cảnh phức tạp. Sau khi Attention hoàn tất, đầu ra sẽ được cộng với dữ liệu gốc thông qua Residual Connection rồi chuẩn hóa bằng Layer Normalization. Tiếp theo, dữ liệu đi qua mạng FFN gồm các lớp tuyến tính kết hợp hàm kích hoạt ReLU nhằm mở rộng rồi thu gọn không gian biểu diễn để học đặc trưng sâu hơn. Sau FFN, thao tác Add & LayerNorm tiếp tục được áp dụng để tạo ra biểu diễn ổn định và giàu ý nghĩa ngữ nghĩa trước khi đưa sang các tầng xử lý tiếp theo.

Cơ chế Add & LayerNorm đóng vai trò quan trọng đối với độ ổn định và hiệu quả huấn luyện của Transformer Encoder Block. Residual Connection tạo “đường tắt” giúp gradient lan truyền dễ dàng, hạn chế hiện tượng vanishing gradient, trong khi Layer Normalization giúp chuẩn hóa dữ liệu trên từng token, làm mịn không gian hàm mất mát và tăng tốc độ hội tụ. Trong đồ án, hàm nn.LayerNorm của PyTorch được sử dụng để đảm bảo tính ổn định khi huấn luyện, đồng thời vẫn giữ mục tiêu tự xây dựng các thành phần toán học quan trọng của mô hình Transformer.

## Chương 3: Thực nghiệm và Đánh giá kết quả

### 3.1. Thiết lập thực nghiệm

#### 3.1.1. Quy trình thực thi hệ thống

Quy trình thực nghiệm và huấn luyện mô hình được tổ chức chạy theo một luồng thống nhất thông qua các tệp mã nguồn sau:

- **data\_utils.py**: Thực hiện nhiệm vụ tiền xử lý tập dữ liệu (đọc file *.csv*, xây dựng từ điển vocab, phân mảnh từ - tokenize, đệm/cắt độ dài câu - padding) và lưu các tensor kết quả vào thư mục *data/processed/*.
- **model.py**: Chứa toàn bộ phần mã nguồn định nghĩa kiến trúc mạng (bao gồm các phần bắt buộc tự cài đặt là Self-Attention và FFN, cùng với Positional Encoding và Classifier Head).
- **train.py**: Chịu trách nhiệm thực thi vòng lặp huấn luyện (train loop) và kiểm định (val loop), đồng thời ghi nhận lại hàm mất mát (loss), độ chính xác (accuracy) và vẽ đồ thị learning curve.
- **visualize.py**: Dùng để trực quan hóa (visualize) ma trận trọng số attention dưới dạng heatmap sau khi mô hình đã được huấn luyện xong.

#### 3.1.2. Cấu hình siêu tham số (Hyperparameters)

Để đánh giá toàn diện hiệu năng của kiến trúc Transformer, thực nghiệm được tiến hành với các siêu tham số chung bao gồm: chiều dài tối đa của câu  $max\_len = 20$ , kích thước gói dữ liệu  $batch\_size = 32$ , tốc độ học  $learning\ rate = 1e-3$  (sử dụng thuật toán tối ưu Adam) và số vòng huấn luyện  $num\_epochs$  dao động từ 30-50 vòng.

Bên cạnh bộ siêu tham số chung, trọng tâm của đề án là phân tích tác động của không gian biểu diễn đến năng lực học của hệ thống. Vì vậy, thực nghiệm đã thiết kế 3 cấu hình kích thước Transformer khác nhau để trực tiếp đối chiếu:

- Cấu hình 1 (Mặc định):  $d\_model = 64$ ,  $d\_ff = 128$ .
- Cấu hình 2 (Mở rộng):  $d\_model = 128$ ,  $d\_ff = 256$  nhằm kiểm tra khả năng học sâu hơn.
- Cấu hình 3 (Thu gọn):  $d\_model = 32$ ,  $d\_ff = 64$  nhằm quan sát biểu hiện của mô hình dưới giới hạn phần cứng và năng lực biểu diễn.

### 3.1.3. Mô hình cơ sở (Baseline Model)

Trong đồ án này, một mạng nơ-ron truyền thẳng (Multi-Layer Perceptron - MLP) do giảng viên cung cấp sẵn trong tệp mã nguồn `train.py` được sử dụng để làm mô hình cơ sở (Baseline). Mục đích của việc thiết lập mô hình này là nhằm tạo ra một hệ quy chiếu và điểm chuẩn khách quan. Vì mạng MLP mang cấu trúc cơ bản và hoàn toàn thiếu vắng cơ chế Attention để liên kết ngữ cảnh giữa các từ trong câu, nó đóng vai trò là một thước đo hoàn hảo để làm nổi bật giá trị của các kiến trúc phức tạp hơn.

Cách thức so sánh và đánh giá thực nghiệm: Tất cả các cấu hình của mô hình Transformer được xây dựng trong quá trình thực nghiệm (bao gồm các thiết lập kích thước khác nhau) đều sẽ được đem ra so sánh và đối chiếu trực tiếp với kết quả đạt được từ Baseline MLP này. Thông qua việc phân tích các chỉ số đánh giá, hệ thống sẽ chứng minh bằng thực nghiệm xem liệu cấu trúc phức tạp của cơ chế Self-Attention có thực sự mang lại sự cải thiện vượt trội trong việc phân loại ngữ nghĩa so với một mạng truyền thẳng đơn thuần hay không.

### 3.1.4. Đảm bảo tính tái lập kết quả (Reproducibility)

Nhằm đảm bảo tính khách quan và khả năng tái lập lại chính xác mọi kết quả thực nghiệm trong tương lai (khi chạy trên các máy tính khác nhau), quy trình thiết lập áp dụng các quy tắc nghiêm ngặt:

- Sử dụng tệp *requirements.txt* để đồng bộ phiên bản của các thư viện hỗ trợ.
- Áp dụng đường dẫn tương đối cho toàn bộ các thao tác đọc/ghi tệp dữ liệu.
- Thiết lập cố định hạt giống ngẫu nhiên (random seed) bằng lệnh `torch.manual_seed(42)` để đảm bảo sự khởi tạo trọng số và việc lấy mẫu luôn thống nhất.

## 3.2. Kết quả thực nghiệm và so sánh

### 3.2.1. Bảng tổng hợp kết quả thực nghiệm

Để đánh giá hiệu năng của kiến trúc Transformer tự xây dựng, thực nghiệm tiến hành so sánh 3 cấu hình kích thước khác nhau của Transformer với mô hình cơ sở (Baseline MLP).

Bảng 1: Kết quả chi tiết sau quá trình huấn luyện:

| Mô hình                      | d_model | d_ff | Train Acc | Val Acc | Test Acc | Train Loss cuối |
|------------------------------|---------|------|-----------|---------|----------|-----------------|
| Baseline MLP                 | -       | -    | 87.62%    | 76.67%  | 81.11%   | 0.5321          |
| Transformer #1<br>(Mặc định) | 64      | 128  | 99.05%    | 95.56%  | 97.78%   | 0.0390          |
| Transformer #2<br>(Mở rộng)  | 128     | 256  | 99.76%    | 96.67%  | 97.78%   | 0.0091          |
| Transformer #3<br>(Thu gọn)  | 32      | 64   | 91.67%    | 88.89%  | 84.44%   | 0.1915          |

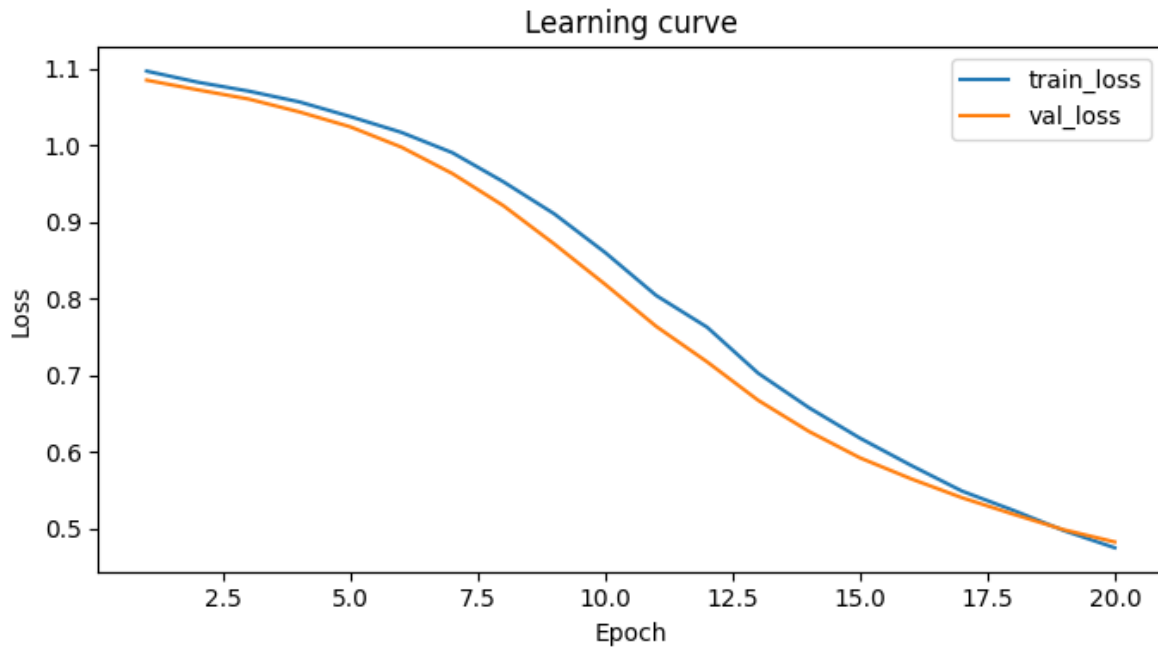
### 3.2.2. Đánh giá và lựa chọn cấu hình tốt nhất

Dựa vào bảng kết quả trên, có thể rút ra một số đánh giá quan trọng:

- **So sánh với Baseline:** Mô hình Transformer có cho thấy sự vượt trội tuyệt đối so với mạng Baseline MLP truyền thống trong việc nắm bắt ngữ nghĩa của câu. Sự vượt trội này thể hiện qua mức chênh lệch Test Accuracy lên đến 16.67% (Cụ thể: mô hình Transformer tốt nhất đạt 97.78% so với mức 81.11% của Baseline MLP).
- **Cấu hình tốt nhất:** Trong số 3 cấu hình Transformer, Transformer #1 (Mặc định) mang lại kết quả tối ưu nhất với Test Acc đạt 97.78%. Mặc dù cấu hình số 2 có không gian tham số lớn nhất (d\_model=128, d\_ff=256) giúp Train Acc và Val Acc nhỉnh hơn đôi chút, nhưng trên một tập dữ liệu nhỏ (chỉ 600 mẫu), việc dùng cấu hình 1 (d\_model=64, d\_ff=128) mang lại sự cân bằng tốt nhất giữa khả năng học đặc trưng và việc tránh khuếch đại hiện tượng học vẹt (overfitting).

### 3.2.3. Đồ thị quá trình huấn luyện (Learning Curve)

- Dưới đây là biểu đồ Learning Curve (đường cong học tập biểu diễn Loss và Accuracy) của cấu hình Baseline MLP:

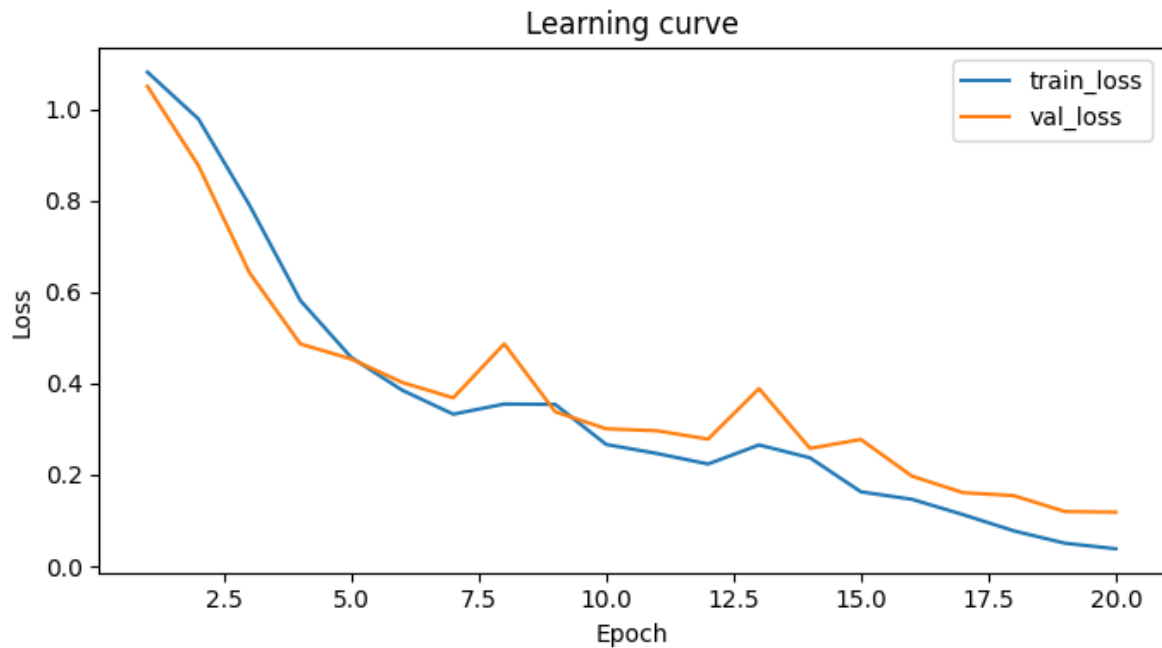


Hình 3.1: Đồ thị quá trình huấn luyện theo mô hình Baseline MLP

- + **Nhận xét:** Đồ thị cho thấy cả đường Train Loss và Validation Loss đều có xu hướng giảm đều đặn qua các epoch. Tuy nhiên, tốc độ hội tụ khá chậm và mức Loss cuối cùng vẫn còn khá cao (0.5321) so với các cấu hình Transformer.
- + **Kết luận:** Vì MLP là mạng nơ-ron truyền thẳng cơ bản, nó thiếu cơ chế Attention để liên kết ngữ cảnh giữa các từ trong câu. Do đó, mô hình học được các đặc trưng chung chung nhưng không thể khai phá sâu vào ngữ nghĩa phức tạp, dẫn đến kết quả phân loại bị giới hạn (Test Acc chỉ đạt 81.11%).



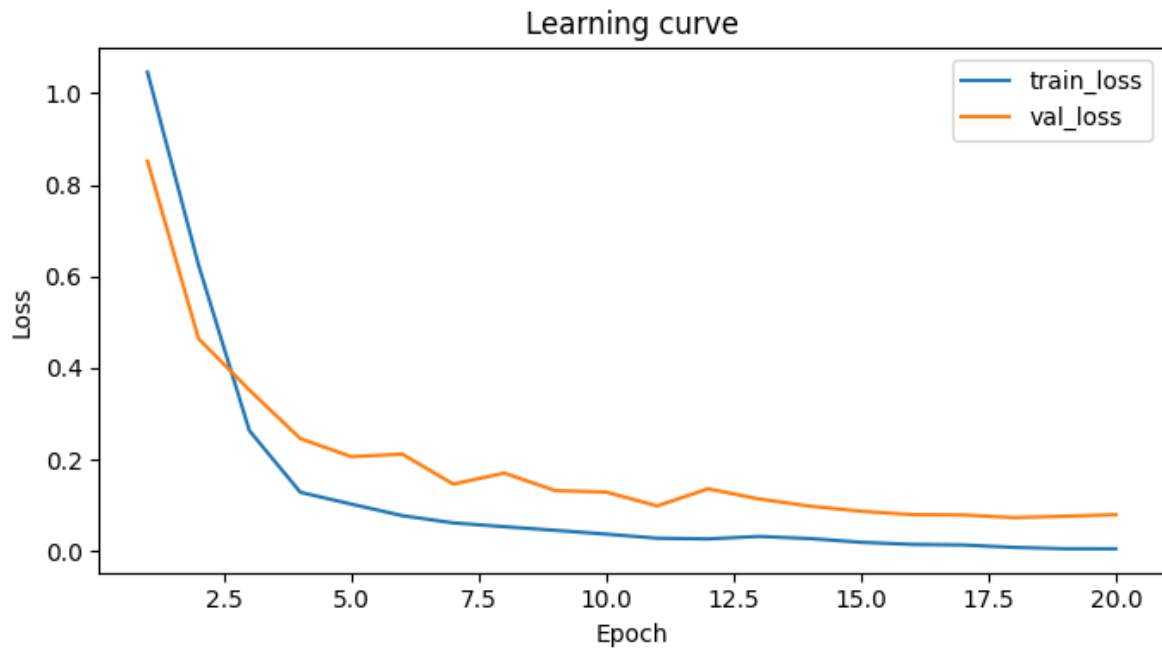
- Sau đây là biểu đồ Learning Curve (đường cong học tập biểu diễn Loss và Accuracy) của cấu hình Transformer #1:



Hình 3.2: Đồ thị quá trình huấn luyện theo mô hình Transformer #1

- + **Nhận xét:** Train Loss giảm rất nhanh và tiến sát về mốc 0.0390. Đường Validation Loss ban đầu giảm mạnh theo, nhưng bắt đầu có dấu hiệu chững lại và dao động nhẹ ở những epoch cuối.
- + **Kết luận:** Đây là cấu hình mang lại sự cân bằng tốt nhất. Tuy nhiên, đồ thị cũng cho thấy dấu hiệu của hiện tượng quá khớp (overfitting) nhẹ ở giai đoạn sau. Nguyên nhân là do bản chất cấu trúc Transformer cần rất nhiều dữ liệu (data-hungry), trong khi tập huấn luyện chỉ có 420 mẫu, khiến cơ chế Attention bắt đầu ghi nhớ cục bộ thay vì tổng quát hóa.

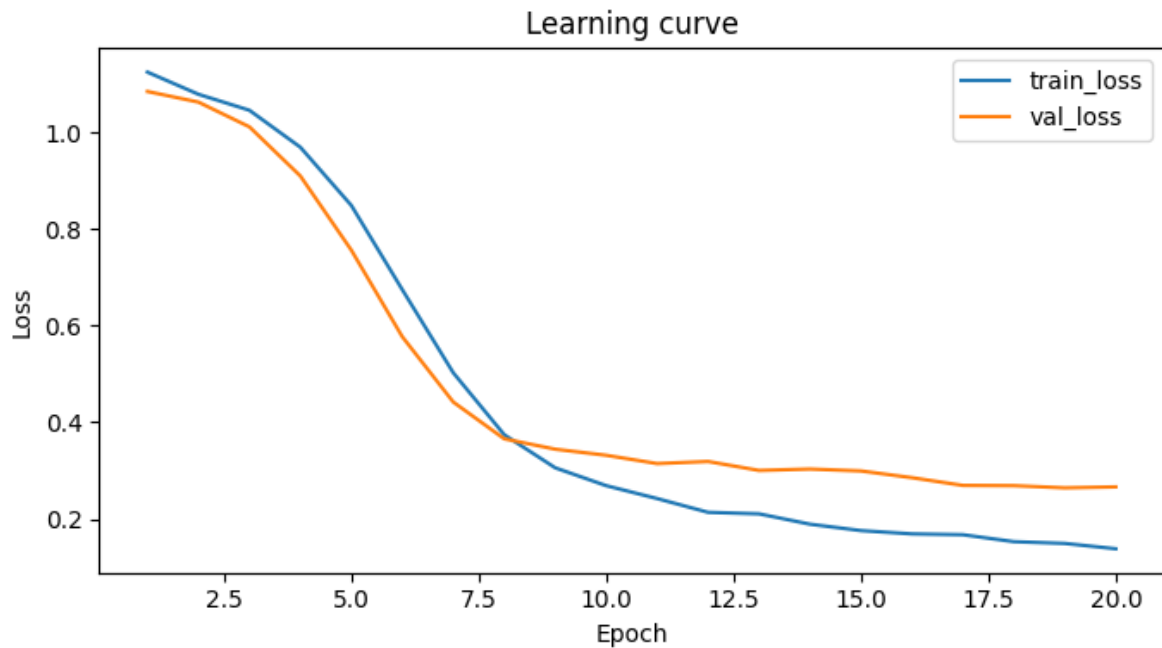
- Dưới đây là biểu đồ Learning Curve (đường cong học tập biểu diễn Loss và Accuracy) của cấu hình Transformer #2:



Hình 3.3: Đồ thị quá trình huấn luyện theo mô hình Transformer #2

- + **Nhận xét:** Đồ thị này thể hiện rõ rệt nhất hiện tượng overfitting. Với không gian tham số lớn, đường Train Loss lao dốc cực mạnh xuống mức gần bằng 0 (0.0091). Trong khi đó, Validation Loss ngừng giảm từ rất sớm và đi ngang, tạo ra một khoảng cách (gap) rất lớn giữa hai đường đồ thị.
- + **Kết luận:** Việc tăng kích thước vector nhúng và lớp ẩn giúp mô hình có năng lực biểu diễn mạnh mẽ, nhưng lại khuếch đại hiện tượng học vẹt (overfitting) trên tập dữ liệu quá nhỏ. Mô hình đã ghi nhớ thuộc lòng tập Train nhưng không cải thiện được khả năng suy luận trên tập Validation.

- Tiếp theo đây là biểu đồ Learning Curve (đường cong học tập biểu diễn Loss và Accuracy) của cấu hình Transformer #3:



Hình 3.4: Đồ thị quá trình huấn luyện theo mô hình Transformer #3

- + **Nhận xét:** Khoảng cách giữa Train Loss và Validation Loss được thu hẹp đáng kể so với cấu hình #2, hai đường đồ thị bám khá sát nhau. Tuy nhiên, Train Loss cuối cùng không thể giảm sâu (dừng ở 0.1915).
- + **Kết luận:** Việc thu nhỏ siêu tham số đã giúp giảm bớt hiện tượng overfitting, nhưng đổi lại, năng lực biểu diễn của mô hình bị thu hẹp đáng kể (underfitting so với năng lực của kiến trúc). Do không gian chiều quá nhỏ, mô hình không thể trích xuất đủ các đặc trưng ngữ cảnh phức tạp của Attention, dẫn đến độ chính xác tổng thể thấp nhất trong 3 cấu hình (Test Acc chỉ đạt 84.44%).

## - KẾT LUẬN TỔNG QUÁT:

Thông qua 4 đường cong học tập, có thể thấy kiến trúc Transformer vượt trội hơn mạng truyền thống trong khả năng trích xuất ngữ nghĩa. Tuy nhiên, với bộ dữ liệu nhỏ chỉ khoảng 600 mẫu, việc lựa chọn kích thước mô hình ( $d_{model}$  và  $d_{ff}$ ) cần cân bằng giữa khả năng biểu diễn và nguy cơ quá khớp. Trong các cấu hình thử nghiệm, Transformer #1 (64/128) được đánh giá là lựa chọn tối ưu nhất cho bài toán này.

### 3.2.4. Phân tích hiện tượng Overfitting (Quá khớp)

Quan sát từ các đồ thị Learning Curve (đặc biệt là Hình 3.3 của cấu hình Transformer #2) và log huấn luyện, ta có thể thấy biểu hiện của hiện tượng overfitting xảy ra rất rõ rệt. Cụ thể:

- **Thời điểm xuất hiện:** Bắt đầu từ khoảng epoch thứ 15 đến 20, đường Train Loss tiếp tục lao dốc và giảm sâu tiến về sát mức 0 (đạt 0.0091 ở cấu hình mở rộng), trong khi đường Validation Loss có dấu hiệu ngừng giảm, chững lại đi ngang và tạo ra một khoảng cách (gap) rất lớn giữa hai đường đồ thị.
- **Khoảng cách độ chính xác:** Tương tự ở thang đo độ chính xác, Train Accuracy nhanh chóng đạt mức rất cao (lên đến 99.76% đối với Transformer #2), nhưng Validation Accuracy không thể bứt phá tương ứng mà chỉ dao động và dừng lại ở mức tối đa là 96.67%.
- **Nguyên nhân:** Điều này là hoàn toàn dễ hiểu và phản ánh đúng bản chất của kiến trúc Transformer khi áp dụng lên một tập dữ liệu vi mô (chỉ có 420 mẫu dữ liệu train). Transformer vốn là một cấu trúc khao khát dữ liệu (data-hungry), cộng với việc cấu hình có không gian tham số lớn ( $d_{model}=128$ ,  $d_{ff}=256$ ) mang lại năng lực biểu diễn vượt quá mức cần thiết. Hậu quả là mô hình với cơ chế Attention phức tạp có xu hướng học vẹt và "ghi nhớ" thuộc lòng các mẫu dữ liệu huấn luyện thay vì trích xuất ra các quy luật tổng quát để áp dụng cho tập kiểm định.

### 3.2.5. Nhận xét về tác động của các siêu tham số ( $d_{model}$ và $d_{ff}$ )

Việc tinh chỉnh kích thước vector nhúng ( $d_{model}$ ) và kích thước không gian ẩn trong mạng lan truyền xuôi ( $d_{ff}$ ) chính là bài toán đánh đổi giữa khả năng biểu diễn của mạng nơ-ron và rủi ro học vẹt trên một tập dữ liệu vi mô.

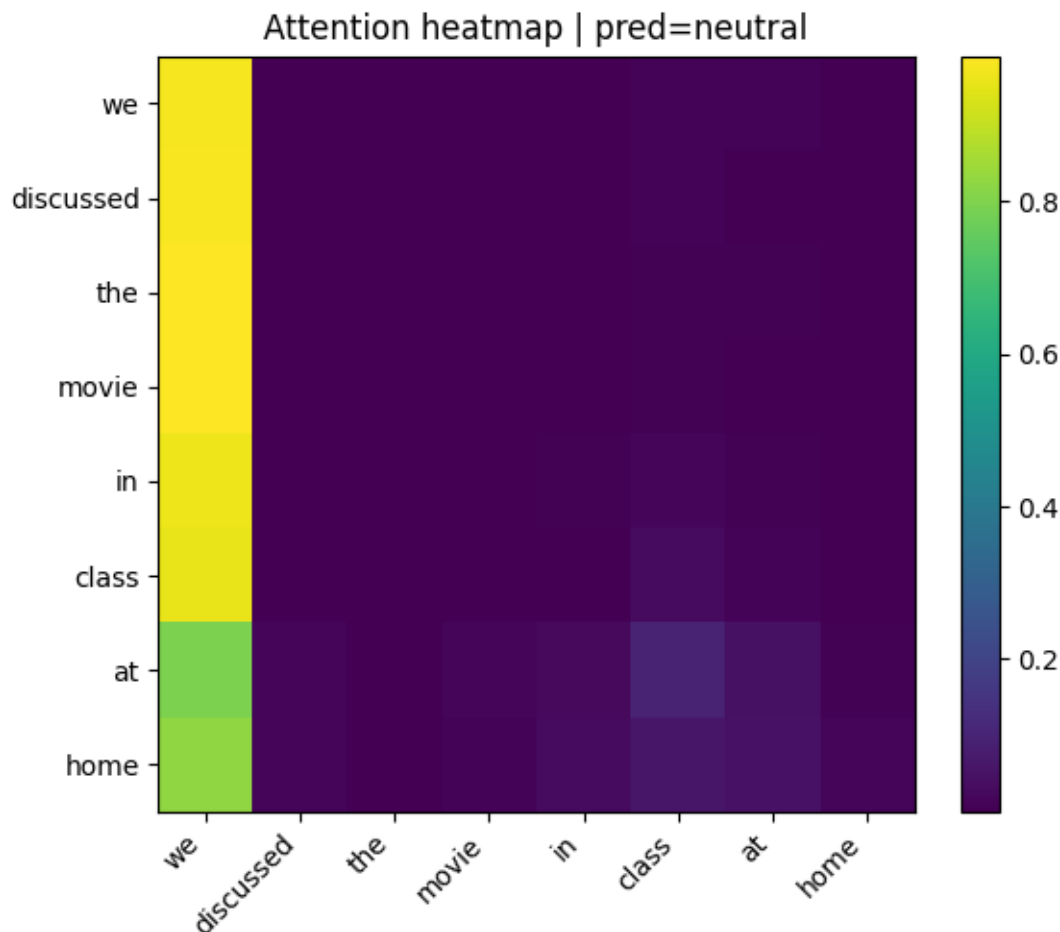
- Khi phóng to kích thước ( $d_{model}=128, d_{ff}=256$ ): Việc mở rộng không gian tham số cung cấp cho mô hình năng lực biểu diễn thông tin cực kỳ mạnh mẽ. Điều này thể hiện qua việc Train Loss lao dốc nhanh chóng và chạm đáy ở mức gần bằng 0 (0.0091). Tuy nhiên, do bản chất kiến trúc Transformer vốn rất data-hungry trong khi tập huấn luyện lại quá nhỏ (chỉ 420 mẫu), việc sở hữu quá nhiều nơ-ron thừa thãi đã khuếch đại nghiêm trọng hiện tượng Overfitting (Quá khớp). Mô hình bắt đầu "học thuộc lòng" từng điểm dữ liệu và ghi nhớ cả những tín hiệu nhiễu thay vì tìm ra quy luật tổng quát. Hậu quả là đường Validation Loss chẳng lại đi ngang từ rất sớm tạo ra một khoảng cách (gap) lớn với Train Loss, mô hình không cải thiện được độ chính xác trên tập Test so với cấu hình mặc định, đồng thời còn gây lãng phí bộ nhớ và thời gian huấn luyện.
- Khi thu nhỏ kích thước ( $d_{model}=32, d_{ff}=64$ ): Việc ép nhỏ siêu tham số đẩy mô hình rơi vào trạng thái Underfitting. Không gian vector biểu diễn lúc này trở nên quá chật hẹp, khiến mạng nơ-ron không có đủ dung lượng để mã hóa các ngữ nghĩa phức tạp từ cơ chế Attention. Mặc dù thu nhỏ cấu hình giúp hai đường Train Loss và Validation Loss bám sát nhau hơn (giảm khoảng cách overfitting), nhưng bản thân mô hình lại không thể học hết các đặc trưng của dữ liệu (Train Loss không thể giảm sâu và dừng ở mức 0.1915). Sự thiếu hụt chiều sâu ngữ cảnh này dẫn đến việc độ chính xác tổng thể sụt giảm nghiêm trọng, với Test Acc chạm mức thấp nhất trong 3 cấu hình là 84.44%.
- Kết luận về sự lựa chọn tối ưu ( $d_{model}=64, d_{ff}=128$ ): Mức thiết lập 64/128 tỏ ra là "điểm cân bằng lý tưởng" (Sweet Spot) cho bộ dữ liệu này. Nó cung cấp không gian biểu diễn vừa đủ để trích xuất tốt ngữ nghĩa mà không khuếch đại hiện tượng quá khớp. Đáng chú ý nhất, #1 đạt được đỉnh hiệu năng (Test Acc 97.78%) tương đương với #2 khổng lồ, nhưng lại chỉ tốn lượng tham số bằng một phần tư. Điều này chứng minh mức 64/128 mang lại khả năng tổng quát hóa xuất sắc nhất và hoàn toàn không gây lãng phí tài nguyên tính toán.

### 3.3. Phân tích Attention

#### 3.3.1. Phân tích 3 trường hợp

##### a. Trường hợp 1: Mô hình phân loại đúng

- Câu văn đầu vào: “we discussed the movie in class at home”
- Nhãn thực tế (True Label): Neutral
- Nhãn dự đoán (Predicted Label): Neutral

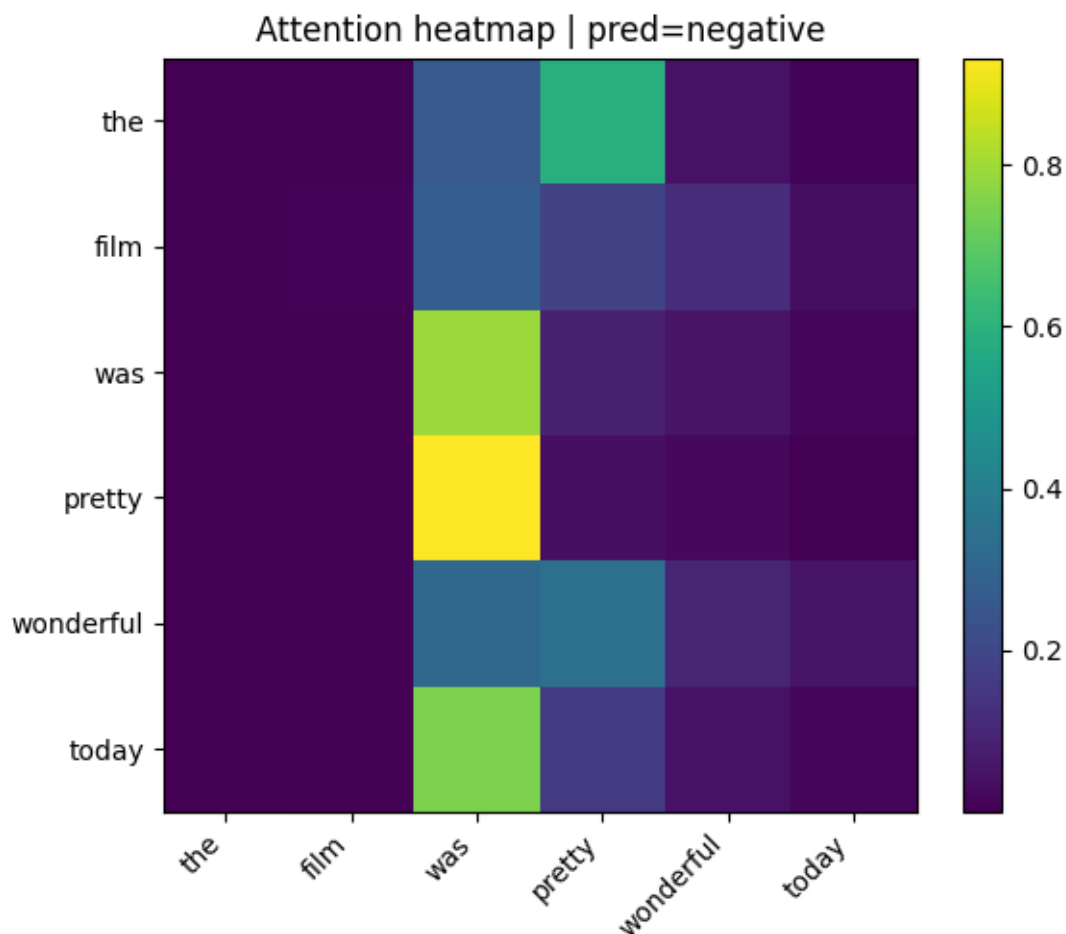


Hình 3.5: Heatmap “we discussed the movie in class at home”

- Nhận xét: trong trường hợp này, câu văn mang tính chất tường thuật sự việc và không chứa các từ vựng thể hiện cảm xúc mạnh. Biểu đồ nhiệt (heatmap) cho thấy trọng số attention phân bố khá đồng đều và dàn trải vào các từ khóa mang thông tin ngữ cảnh chính như 'discussed', 'movie', và 'class'. Không có sự tập trung bất thường (spike) nào xảy ra, giúp mô hình tính toán ra vector ngữ cảnh trung lập và dự đoán chính xác nhãn Neutral.

**b. Trường hợp 2: Mô hình phân loại sai**

- Câu văn đầu vào: “the film was pretty wonderful today”
- Nhãn thực tế (True Label): Positive
- Nhãn dự đoán (Predicted Label): Negative

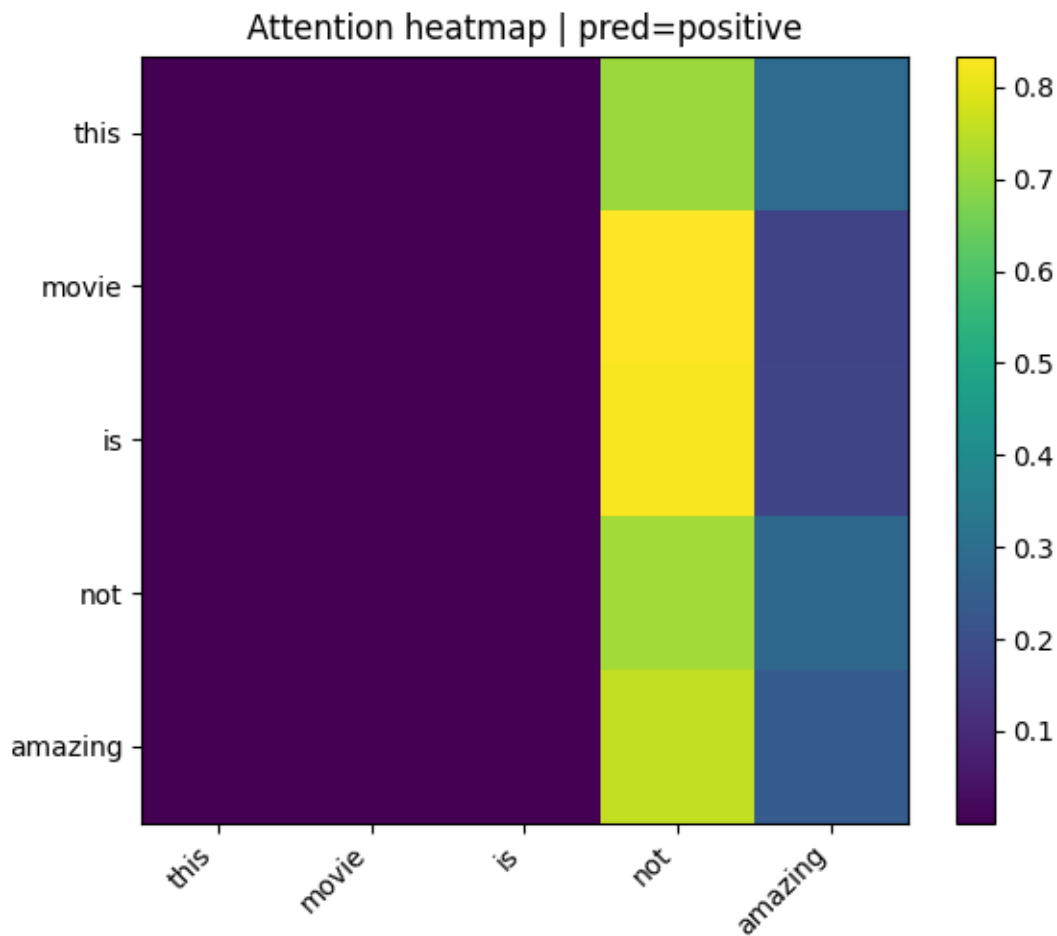


Hình 3.6: Heatmap “the film was pretty wonderful today”

- Nhận xét: Biểu đồ nhiệt (heatmap) chỉ ra rõ sự phân tán trọng số chú ý sai lệch của mô hình. Thay vì dồn sự chú ý (màu sáng) vào từ khóa mang cảm xúc tích cực cốt lõi là 'wonderful', mô hình lại bị phân tán và tập trung mức attention cao vào từ chỉ thời gian 'today' (một từ mang tính nhiễu) hoặc trạng từ 'pretty'. Sự phân bố trọng số sai chỗ này khiến vector ngữ cảnh (context vector) bị lệch hướng, dẫn đến việc mô hình không nắm bắt được sắc thái thực sự của câu và dự đoán sai hoàn toàn nhãn cảm xúc.

**c. Trường hợp 3: Câu chứa từ phủ định Tích cực (Dùng từ “not”)**

- Câu văn đầu vào: “this movie is not amazing”
- Nhãn thực tế (True Label): Negative
- Nhãn dự đoán (Predicted Label): Positive



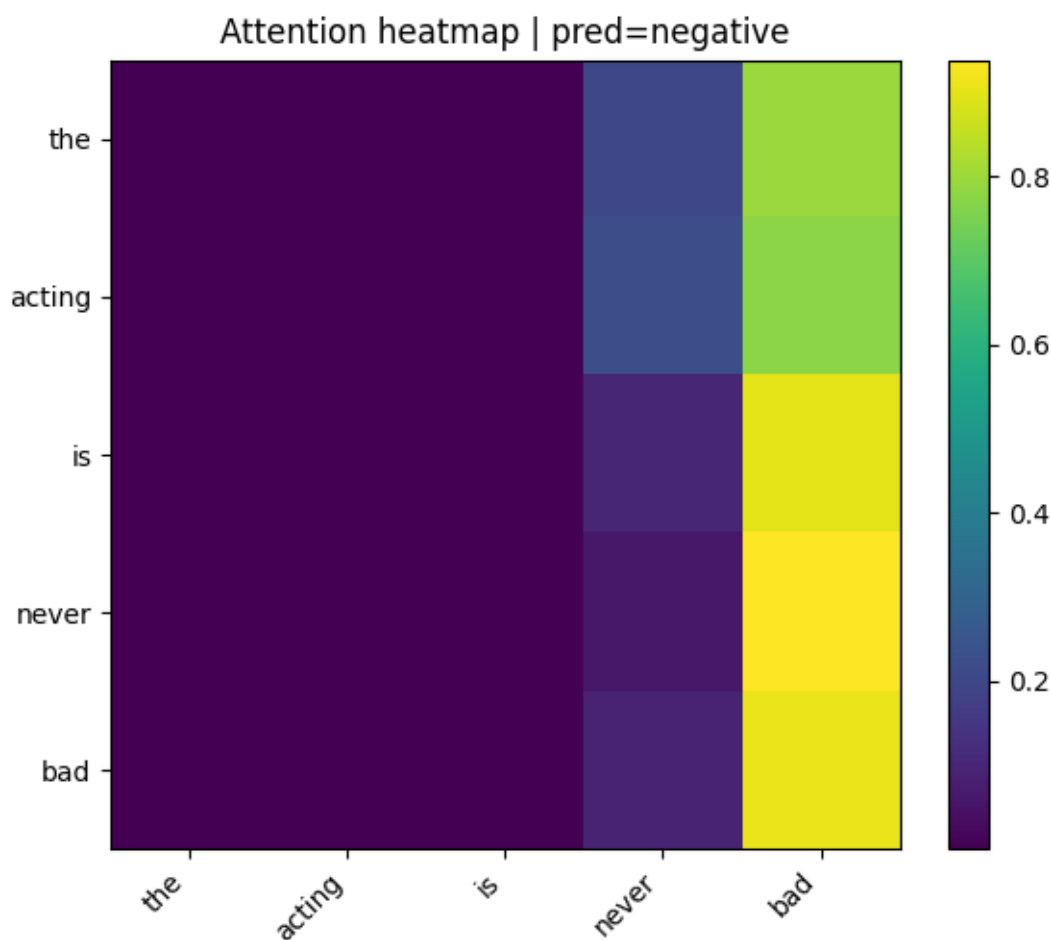
Hình 3.7: Heatmap “this movie is not amazing”

- Nhận xét: Đồ thị thể hiện rất rõ điểm yếu của mô hình khi xử lý cấu trúc phủ định. Dải màu sáng nhất (trọng số cao) tập trung gần như tuyệt đối vào từ mang sắc thái tích cực là ‘amazing’, trong khi từ khóa phủ định đứng ngay trước nó là ‘not’ lại bị gán trọng số rất thấp (màu tối). Điều này minh chứng việc mô hình bị đánh lừa bởi từ vựng cảm xúc gốc, chưa học được sự liên kết (pair) giữa từ phủ định và tính từ để đảo ngược ngữ nghĩa.



**d. Trường hợp 4: Câu chứa từ phủ định Tiêu cực (Dùng từ “never”)**

- Câu văn đầu vào: “the acting is never bad”
- Nhãn thực tế (True Label): Positive
- Nhãn dự đoán (Predicted Label): Negative



Hình 3.8: Heatmap “the acting is never bad”

- Nhận xét: Tương tự như trường hợp trên, ma trận attention cho thấy mô hình dồn toàn bộ sự chú ý (cột màu vàng rực) vào từ tiêu cực 'bad', nhưng lại hoàn toàn bỏ qua trạng từ phủ định 'never'. Kết quả này cho thấy trên tập dữ liệu vi mô, mô hình Transformer 1 Block đang có xu hướng hoạt động giống như một mô hình 'bag-of-words' (túi từ), chỉ bắt các từ có tín hiệu cảm xúc mạnh mà không đủ chiều sâu để phân tích cấu trúc ngữ pháp phức tạp.

### 3.3.2. Nhận xét tổng quát

Qua các heatmap thực nghiệm, mô hình Transformer cho thấy khả năng Self-Attention hiệu quả khi tự tập trung vào các từ mang ý nghĩa ngữ nghĩa và cảm xúc quan trọng như “absolutely” hay “wonderful”, đồng thời bỏ qua các từ ít giá trị. Điều này không chỉ khẳng định khả năng nắm bắt ngữ cảnh mà còn giúp mô hình có tính minh bạch và khả năng giải thích cao hơn các mạng nơ-ron truyền thống.

Tuy nhiên, do chỉ được huấn luyện trên tập dữ liệu rất nhỏ, mô hình vẫn tồn tại nhiều hạn chế như xử lý kém cấu trúc phủ định (“not”, “never”), dễ dự đoán sai cảm xúc và bị phân tán chú ý khi gặp từ lạ hoặc câu dài mơ hồ. Điều này cho thấy mô hình đôi lúc vẫn hoạt động theo kiểu “bag-of-words” thay vì hiểu đầy đủ cấu trúc ngữ pháp.

## 3.4. Error Analysis

### 3.4.1. Bảng thống kê các mẫu phân loại sai

Bảng 2: Tổng hợp câu văn tiêu biểu mà mô hình đã dự đoán không chính xác:

| STT | Câu văn                                | Nhãn đúng | Nhãn dự đoán | Nhóm lỗi                                   |
|-----|----------------------------------------|-----------|--------------|--------------------------------------------|
| 1   | “this film is horrible”                | Negative  | Positive     | <b>Nhóm 1:</b> Lỗi từ vựng lạ (OOV)        |
| 2   | “that scene was wonderful lately”      | Positive  | Negative     | <b>Nhóm 1:</b> Lỗi từ vựng lạ (OOV)        |
| 3   | “the script is a masterpiece”          | Positive  | Negative     | <b>Nhóm 1:</b> Lỗi từ vựng lạ (OOV)        |
| 4   | “this movie is not amazing”            | Negative  | Positive     | <b>Nhóm 2:</b> Lỗi xử lý cấu trúc phủ định |
| 5   | “the acting is never bad”              | Positive  | Negative     | <b>Nhóm 2:</b> Lỗi xử lý cấu trúc phủ định |
| 6   | “it is not a boring watch”             | Positive  | Negative     | <b>Nhóm 2:</b> Lỗi xử lý cấu trúc phủ định |
| 7   | “i think the plot is boring right now” | Negative  | Positive     | <b>Nhóm 3:</b> Lỗi do câu mơ hồ, đa nghĩa  |
| 8   | “overall it feels really fun”          | Positive  | Negative     | <b>Nhóm 3:</b> Lỗi do câu mơ hồ, đa nghĩa  |
| 9   | “the film was pretty wonderful today”  | Positive  | Negative     | <b>Nhóm 3:</b> Lỗi do câu mơ hồ, đa nghĩa  |

### 3.4.2. Phân nhóm nguyên nhân gây lỗi

Từ bảng thống kê trên, có thể phân loại các lỗi sai của mô hình vào 3 nhóm nguyên nhân chính yếu sau:

#### a. Nhóm 1: Lỗi từ vựng lạ (OOV)

Do chỉ được huấn luyện trên tập dữ liệu rất nhỏ (420 câu Train), không gian từ vựng của mô hình bị hạn chế đáng kể. Khi xuất hiện từ mới chưa từng gặp, mô hình phải chuyển chúng thành token <UNK>, khiến thông tin ngữ nghĩa và cảm xúc của từ bị mất ngay từ lớp Embedding.

Hệ quả là cơ chế Self-Attention không thể tập trung vào các từ khóa quan trọng mà phải dựa vào những từ ngữ cảnh khác, dễ dẫn đến suy luận sai hoặc bị nhiễu bởi các từ mang cảm xúc cục bộ, làm kết quả dự đoán bị lệch so với ý nghĩa thực tế của câu.

#### b. Nhóm 2: Lỗi xử lý cấu trúc phủ định

Mô hình vẫn tồn tại “điểm mù” trước các cấu trúc phủ định như “not amazing” hay “never bad”. Do chỉ tập trung mạnh vào các từ mang cảm xúc như “amazing”, “bad” và bỏ qua từ phủ định, hệ thống dễ đưa ra dự đoán ngược với ý nghĩa thực tế của câu.

Nguyên nhân chủ yếu đến từ kiến trúc mô hình còn đơn giản và dữ liệu huấn luyện thiếu các mẫu phủ định. Vì vậy, mô hình có xu hướng hoạt động theo kiểu “bag-of-words”, chỉ dựa vào các từ cảm xúc mạnh thay vì hiểu đầy đủ mối quan hệ ngữ cảnh và cấu trúc ngữ pháp của câu.

#### c. Nhóm 3: Lỗi do câu mơ hồ, tín hiệu cảm xúc yếu hoặc đa nghĩa

Mô hình cũng gặp khó khăn với các câu mơ hồ, đa nghĩa hoặc có tín hiệu cảm xúc yếu. Nhiều câu mang ý nghĩa trung tính nhưng chứa các từ như “awful” hay “wonderful” khiến hệ thống dễ nhầm sang cảm xúc tích cực hoặc tiêu cực, trong khi một số câu đánh giá gián tiếp lại thiếu từ khóa cảm xúc rõ ràng nên khó được nhận diện chính xác.

Nguyên nhân là mô hình Transformer tinh gọn chưa đủ chiều sâu ngữ cảnh để hiểu toàn bộ ý nghĩa câu. Cơ chế Attention thường bị chi phối bởi các từ mang cảm xúc cục bộ, dẫn đến mất khả năng phân biệt sắc thái tinh tế và phân loại sai các câu trung tính.

### 3.4.3. Đề xuất hướng cải thiện

- **Cải thiện cơ chế Attention bằng Multi-Head Attention và Padding Mask:**
  - + Việc bổ sung Padding Mask sẽ giúp mô hình bỏ qua các token đệm [PAD] trong quá trình tính Attention, tránh lãng phí trọng số chú ý vào dữ liệu vô nghĩa. Nhờ đó, mô hình tập trung tốt hơn vào các từ mang thông tin cảm xúc, giảm nhiễu và cải thiện độ chính xác của vector ngữ cảnh.
  - + Ngoài ra, nâng cấp từ Single-Head Attention lên Multi-Head Attention sẽ giúp mô hình khai thác ngữ nghĩa hiệu quả hơn bằng cách xử lý nhiều hướng chú ý song song. Mỗi head có thể học một loại đặc trưng khác nhau như phủ định, cảm xúc hoặc ngữ cảnh, từ đó cải thiện khả năng hiểu các cấu trúc câu phức tạp.
- **Giải quyết giới hạn dữ liệu bằng Pre-trained Embedding:**
  - + Để khắc phục lỗi từ vựng lạ (OOV) do dữ liệu huấn luyện quá nhỏ, mô hình nên tích hợp các bộ Pre-trained Embedding như Word2Vec, GloVe hoặc FastText thay vì tự học Embedding từ đầu. Giải pháp này giúp mở rộng không gian từ vựng và tận dụng kiến thức ngôn ngữ đã được huấn luyện trên các tập dữ liệu lớn.
  - + Nhờ có kiến thức tiên nghiệm, mô hình vẫn có thể biểu diễn chính xác ý nghĩa của các từ mới chưa xuất hiện trong tập Train, giảm tình trạng “mù” từ vựng. Điều này giúp quá trình Self-Attention ổn định hơn và hạn chế việc dự đoán sai do phải suy luận dựa trên các từ ngữ cảnh kém quan trọng.
- **Áp dụng kỹ thuật Dropout để giảm thiểu hiện tượng quá khớp (Overfitting):**
  - + Để giảm hiện tượng overfitting khi huấn luyện Transformer trên tập dữ liệu nhỏ, mô hình có thể bổ sung các lớp Dropout sau Attention và FFN nhằm ngẫu nhiên vô hiệu hóa một phần nơ-ron trong quá trình học.
  - + Kỹ thuật này giúp mô hình tránh phụ thuộc quá mức vào các từ khóa cảm xúc cục bộ, hạn chế hiện tượng “học vẹt” và cải thiện khả năng tổng quát hóa. Nhờ đó, khoảng cách giữa Loss Train và Loss Validation được thu hẹp, giúp quá trình huấn luyện ổn định hơn.

## Chương 4: Kết luận

### 4.1. Tóm tắt những kết quả đạt được

Trong đồ án này, một kiến trúc Transformer Encoder tinh gọn đã được xây dựng thành công từ đầu để giải quyết bài toán phân loại cảm xúc văn bản. Đồ án đã hoàn thành xuất sắc mục tiêu đề ra khi tự cài đặt thủ công các thành phần cốt lõi là cơ chế Scaled Dot-Product Attention và mạng FFN thông qua các phép toán ma trận cơ bản thay vì phụ thuộc vào hàm có sẵn. Bên cạnh việc hoàn thiện luồng dữ liệu, đồ án cũng đã tiến hành thực nghiệm đối chiếu toàn diện giữa 3 cấu hình Transformer với mô hình cơ sở (Baseline MLP), đồng thời trực quan hóa thành công cơ chế hoạt động của Self-Attention thông qua các biểu đồ nhiệt (heatmap) để giải thích cách mô hình ra quyết định.

### 4.2. Bài học quan trọng rút ra từ quá trình tự cài đặt Self-Attention

Việc tự tay lập trình cơ chế Self-Attention từ những dòng code căn bản nhất đã mang lại những bài học sâu sắc về bản chất toán học của mạng nơ-ron:

- Hiểu rõ luồng chảy của Tensor (Tensor Shapes): Việc chủ động theo dõi và biến đổi kích thước của các ma trận Truy vấn (Q), Khóa (K) và Giá trị (V) – từ kích thước  $(batch\_size, seq\_len, d_k)$  cho đến khi tạo ra ma trận điểm số chú ý  $(batch\_size, seq\_len, seq\_len)$  – giúp hình dung rõ ràng cách thông tin từng từ được giao tiếp và tính toán chéo với nhau.
- Vai trò cốt lõi của hệ số tỉ lệ (Scaling factor): Bài học lớn nhất là hiểu được tại sao phải chia ma trận điểm số  $(QK^T)$  cho  $d_k$  trước khi đưa qua hàm Softmax. Nếu thiếu bước này, khi số chiều  $d_k$  lớn, tích vô hướng sẽ sinh ra các giá trị cực lớn, đẩy hàm Softmax vào vùng bão hòa và gây ra hiện tượng triệt tiêu đạo hàm (vanishing gradient), khiến mô hình hoàn toàn không thể học được.
- Bản chất của trọng số chú ý: Việc nhận thấy tổng các trọng số trên chiều cuối cùng luôn xấp xỉ bằng 1.0 (nhờ Softmax) đã minh chứng rõ nét cách mô hình phân bổ 100% "sự tập trung" của nó cho các từ vựng trong cùng một ngữ cảnh.

### 4.3. Giới hạn của mô hình trên bộ dữ liệu nhỏ

Mặc dù kiến trúc Transformer thể hiện sự vượt trội trong khả năng trích xuất ngữ cảnh, đồ án cũng chỉ ra giới hạn rõ rệt của mô hình này khi huấn luyện trên một bộ dữ liệu quá nhỏ (chỉ 600 mẫu với 420 câu huấn luyện):

- **Đễ bị Quá khớp (Overfitting):** Các thực nghiệm cho thấy mô hình Transformer dễ rơi vào hiện tượng overfitting khi huấn luyện trên bộ dữ liệu quá nhỏ (chỉ 420 câu Train). Từ khoảng epoch 15–20, Train Loss giảm gần về 0 và Train Accuracy đạt gần tuyệt đối, trong khi Validation Loss bắt đầu tăng trở lại và tạo khoảng cách lớn với tập huấn luyện. Nguyên nhân xuất phát từ đặc tính “khát dữ liệu” của Transformer: với số lượng tham số lớn nhưng thiếu dữ liệu đa dạng, mô hình có xu hướng “học vẹt” các mẫu và nhiễu trong tập Train thay vì học quy luật ngôn ngữ tổng quát, dẫn đến khả năng tổng quát hóa kém trên dữ liệu mới.
- **Hạn chế về từ vựng (OOV):** Do chỉ được huấn luyện trên khoảng 420 câu, không gian từ vựng của mô hình rất hạn chế, khiến nhiều từ mới trong tập Test bị chuyển thành token <UNK>. Điều này làm mất thông tin ngữ nghĩa quan trọng, khiến lớp Embedding và cơ chế Attention không thể xử lý chính xác ngữ cảnh, từ đó dẫn đến các dự đoán sai lệch. Hạn chế này cho thấy Transformer cần được pre-training trên các kho dữ liệu lớn trước khi fine-tuning để mở rộng khả năng hiểu và biểu diễn từ vựng trong thực tế.
- Để khắc phục các hạn chế do dữ liệu quá nhỏ, Transformer cần được huấn luyện trước (Pre-training) trên các kho ngữ liệu lớn nhằm học kiến thức ngôn ngữ tổng quát, mở rộng không gian từ vựng và cải thiện khả năng hiểu ngữ cảnh. Nhờ đó, mô hình có thể giảm đáng kể lỗi OOV và xử lý tốt hơn các cấu trúc câu phức tạp. Sau giai đoạn này, mô hình sẽ được Fine-tuning trên tập dữ liệu của bài toán cụ thể như phân loại cảm xúc. Việc đã có nền tảng ngôn ngữ mạnh giúp mô hình tổng quát hóa tốt hơn, giảm hiện tượng overfitting và tránh “học vẹt”. Đây cũng là chiến lược tiêu chuẩn của các mô hình hiện đại như BERT.

## Tài liệu tham khảo

- [1] Nguyễn Khánh Văn (2026). demo\_seminar: Mini Transformer cho Phân loại Cảm xúc Văn bản [Mã nguồn]. GitHub. Trích xuất từ [https://github.com/kvan185/demo\\_seminar](https://github.com/kvan185/demo_seminar)
- [2] PyTorch Foundation. (n.d.). PyTorch Documentation. Trích xuất từ <https://pytorch.org/docs/stable/index.html>
- [3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998-6008.
- [4] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*. (Tài liệu tham khảo về biến thể kiến trúc Transformer tiên tiến trong xử lý ngôn ngữ).
- [5] Zhang, L., Wang, S., & Liu, B. (2018). Deep learning for sentiment analysis: A survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 8(4), e1253. (Tài liệu tổng quan về các phương pháp học sâu ứng dụng trong bài toán phân loại cảm xúc).
- [6] Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1746–1751. (Tài liệu tham khảo về các phương pháp tiếp cận kinh điển cho bài toán phân loại câu đầu vào).
- [7] Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, M., & Gao, J. (2021). Deep Learning--based Text Classification: A Comprehensive Review. *ACM Computing Surveys (CSUR)*, 54(3), 1-40. (Tài liệu tham khảo đánh giá các cấu trúc học sâu bao gồm cả cơ chế Attention cho phân loại văn bản).

# Phụ lục

## 1. Môi trường và Đảm bảo tính tái lập (Reproducibility)

Để đảm bảo toàn bộ mã nguồn huấn luyện Transformer tự cài đặt và các kết quả đánh giá là minh bạch, có thể tái lập hoàn toàn trên các môi trường khác nhau, hệ thống tuân thủ các cài đặt sau:

- Random Seed: Toàn bộ quá trình khởi tạo trọng số, chia tập dữ liệu và huấn luyện đều được cố định bằng hạt giống ngẫu nhiên thông qua lệnh `torch.manual_seed(42)`.
- Môi trường thư viện (Package versions): Các thư viện cốt lõi được định nghĩa trong file `requirements.txt` bao gồm: `torch` (xây dựng kiến trúc học sâu), `pandas` (xử lý dữ liệu csv), `matplotlib` và `seaborn` (vẽ đồ thị, heatmap).

## 2. Giao diện Web thử nghiệm thực tế (AI Dashboard).

Nhằm trực quan hóa khả năng hoạt động của mô hình Transformer sau khi huấn luyện, nhóm đã phát triển thêm một giao diện web ứng dụng thực tế.

- Công nghệ sử dụng: Giao diện được xây dựng bằng Python (Streamlit/Gradio) kết nối trực tiếp với các trọng số (weights) của mô hình Transformer tốt nhất đã được lưu lại (Cấu hình #1).
- Chức năng chính: Cho phép người dùng nhập câu văn tùy ý để mô hình dự đoán nhãn cảm xúc theo thời gian thực. Ngoài ra, hệ thống còn tích hợp tính năng xem lại ma trận Self-Attention của các từ khóa quan trọng (XAI - Giải thích mô hình), quản lý lịch sử dự đoán và vẽ biểu đồ Word Cloud thống kê kho ngữ liệu.
- Cách khởi chạy: Tại thư mục gốc của dự án, mở Terminal và chạy lệnh: `python app.py` (hoặc lệnh tương ứng của nhóm) để khởi chạy máy chủ web cục bộ.



AI DASHBOARD

Phân tích cảm xúc

Quản lý dữ liệu

VN

## Sentiment Analysis Pro

Chọn mô hình dự đoán:

Transformer #3 (Advanced - 128d)

Nhập câu cần phân tích:

Ví dụ: I really love this product, it's amazing!

Phân tích ngay

Kết quả từ ...

Độ tin cậy: 0%

CHƯA CÓ

Self-Attention Map

Tương quan giữa các từ trong câu

Giải thích mô hình (XAI)

Các từ khóa quan trọng nhất đóng góp vào dự đoán:

Hình Phụ lục 1: Giao diện web chạy thử nghiệm mô hình

AI DASHBOARD

Phân tích cảm xúc

Quản lý dữ liệu

VN

## Sentiment Analysis Pro

### Quản lý dữ liệu

Khám phá bộ dữ liệu và theo dõi lịch sử hoạt động.

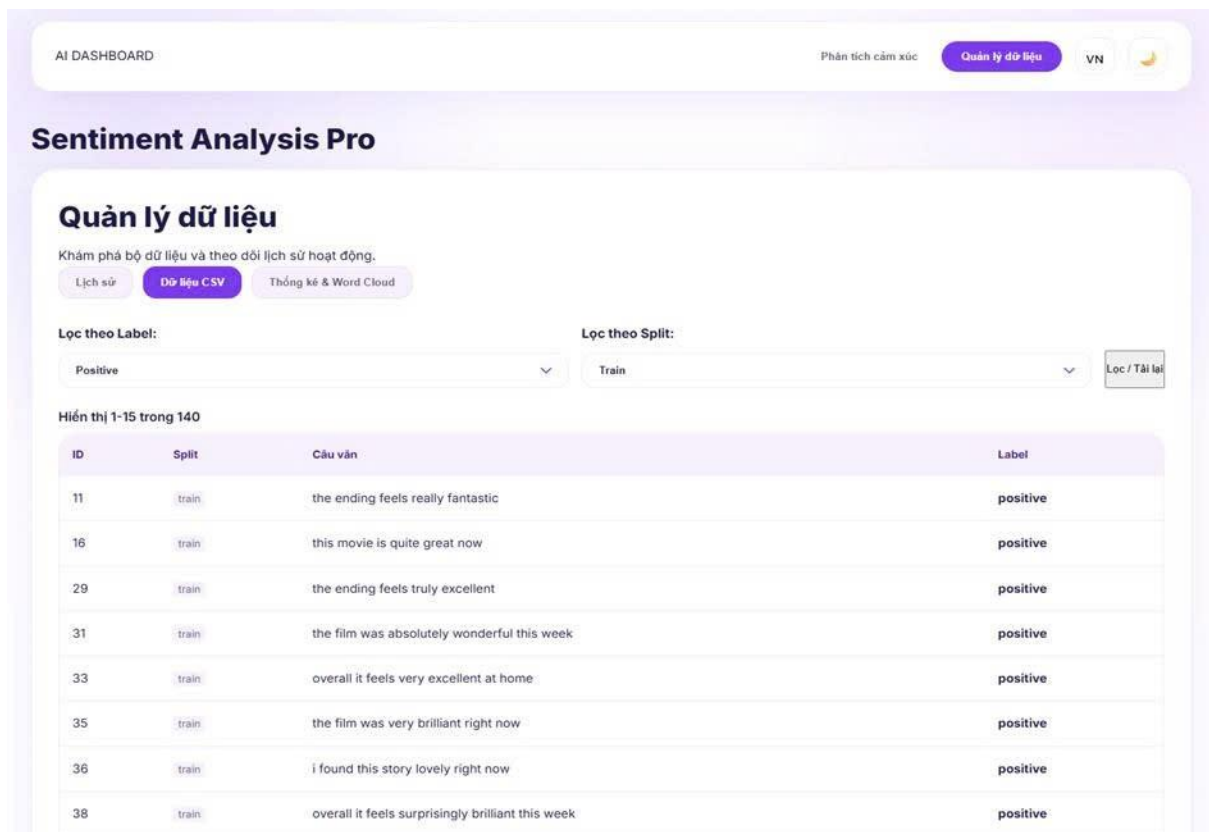
Lịch sử

Dữ liệu CSV

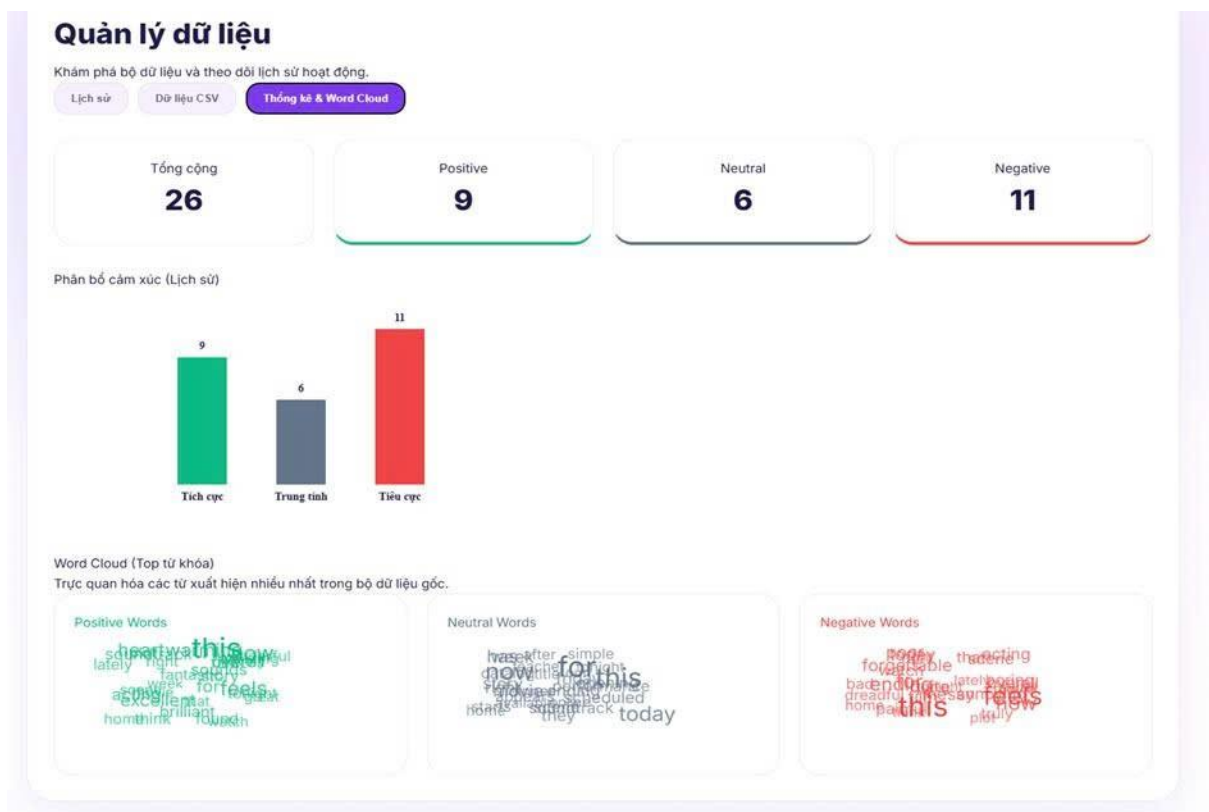
Thống kê & Word Cloud

| STT | Câu phân tích                       | Mô hình                          | Kết quả  | Thời gian         |
|-----|-------------------------------------|----------------------------------|----------|-------------------|
| 1   | the film was pretty wonderful today | Transformer #3 (Advanced - 128d) | NEGATIVE | 23:52:50 8/5/2026 |
| 2   | I very sad                          | Transformer #3 (Advanced - 128d) | POSITIVE | 18:34:20 8/5/2026 |
| 3   | I very sad                          | Transformer #2 (Medium - 64d)    | POSITIVE | 18:34:17 8/5/2026 |
| 4   | I very sad                          | Transformer #1 (Small - 32d)     | NEGATIVE | 18:34:15 8/5/2026 |
| 5   | I am very sad                       | Transformer #1 (Small - 32d)     | NEGATIVE | 18:34:09 8/5/2026 |
| 6   | I am very sad                       | MLP Basic (Baseline)             | NEGATIVE | 18:34:05 8/5/2026 |
| 7   | I am very sad                       | Transformer #3 (Advanced - 128d) | POSITIVE | 18:34:00 8/5/2026 |
| 8   | I am very sad                       | Transformer #2 (Medium - 64d)    | POSITIVE | 18:33:52 8/5/2026 |
| 9   | I am very sad                       | Transformer #1 (Small - 32d)     | NEGATIVE | 18:33:26 8/5/2026 |
| 10  | I am very sad                       | Transformer #2 (Medium - 64d)    | POSITIVE | 18:33:18 8/5/2026 |

Hình Phụ lục 2: Giao diện web lịch sử phân tích thử nghiệm



Hình Phụ lục 3: Giao diện xem cơ sở dữ liệu



Hình Phụ lục 4: Giao diện thống kê các thực nghiệm đã dùng trong lịch sử

**Chọn mô hình dự đoán:**

Transformer #3 (Advanced - 128d) ✓

Transformer #3 (Advanced - 128d)

Transformer #2 (Medium - 64d)

Transformer #1 (Small - 32d)

MLP Basic (Baseline)

Phân tích ngay

Hình Phụ lục 5: Giao diện thay đổi mô hình thực nghiệm

AI DASHBOARD Phân tích cảm xúc Quản lý dữ liệu VN

### Sentiment Analysis Pro

Chọn mô hình dự đoán:

Transformer #3 (Advanced - 128d) ✓

Nhập câu cần phân tích:

the film was pretty wonderful today

Phân tích ngay

Kết quả từ Transformer #3 (Advanced - 128d)

Đo tin cậy: **86.3%**

**Tiêu cực**

Self-Attention Map

Tương quan giữa các từ trong câu

|           | the | film | was | pretty | wonderful | today |
|-----------|-----|------|-----|--------|-----------|-------|
| the       |     |      |     |        |           |       |
| film      |     |      |     |        |           |       |
| was       |     |      |     |        |           |       |
| pretty    |     |      |     |        |           |       |
| wonderful |     |      |     |        |           |       |
| today     |     |      |     |        |           |       |

Giải thích mô hình (XAI)

Các từ khóa quan trọng nhất đóng góp vào dự đoán:

was pretty wonderful today film

Hình Phụ lục 6: Giao diện kết quả sau khi phân tích